

Questions

Questions::

=====

1. Application Overview

=====

- Tell me about your application

Our application is an internal auditing app used by companies to perform audit workflow in the proper manner.

Audit is basically maintaining internal information about an entity

For ex your finance department auditing will include fields like risks ,control ,documents

- Who uses it?

Senior/junior auditors ,directors ,managers

- Why do they use it?

To have an insight of what going on inside the company

=====

2. Architecture & Design

=====

- Architecture of the application

Our application follows a hybrid architecture and is currently in a transition phase from a monolithic system to a microservices-based architecture.

At present, we have a single monolithic application that acts as the primary API Gateway. It serves as the main entry point for client requests and handles request routing, authentication, and some shared business logic.

Alongside the monolith, we have identified and segregated certain business functionalities into independent microservices. These microservices are gradually being extracted from the monolith and communicate with it through REST APIs and asynchronous messaging where required.

On the frontend, we use React with TypeScript for building the user interface, Redux for state management, and Vite as the build tool to ensure fast development and optimized builds.

For infrastructure and supporting components:

- Redis is used for caching and performance optimization.
- Kafka is used for asynchronous event-driven communication.
- Docker is used for containerizing application components.
- Kubernetes is used for container orchestration and auto-scaling.
- AWS EC2 is used for hosting application services.
- AWS Lambda is used for specific event-driven or background tasks.
- AWS S3 is used for secure document and file storage.

- Splunk is used for centralized logging.
- Datadog is used for monitoring, metrics, and alerting.

Currently, the application uses a single shared database, which is typical during the initial phase of migrating from a monolith to microservices. This setup simplifies data consistency and reduces operational complexity while the migration is ongoing.

Overall, this architecture enables us to maintain system stability, support scalability, and modernize the application incrementally without impacting existing users.

- Why this architecture?

We chose this hybrid architecture because the application was originally built as a monolith and was already serving active users. Moving directly to a full microservices architecture would have been risky, time-consuming, and could have impacted production stability.

By keeping the monolith as the central entry point and gradually extracting selected modules into microservices, we can modernize the system incrementally without disrupting existing functionality.

This approach allows us to:

- Avoid a big-bang rewrite
- Reduce production risk
- Continue feature development during migration
- Scale only high-traffic or critical modules as microservices
- Validate microservices in production step by step

n6-

Using a single shared database during the transition simplifies data consistency and reduces operational complexity

Que on Archetecure::

1) which services converted to microservices and why?

-> We first extracted reporting, document management, notification, and background job modules into microservices. These modules were either resource-intensive, event-driven, or required independent scalability.

Core transactional audit workflows remained in the monolith to maintain data consistency and reduce migration risk during the transition phase.

2) how do you debug and which software do you use ?

- Synchronous (restTemplate)**
- Asynchronous(web client)**
- kafka**

- Advantages of the architecture

1. Gradual Migration

- Enables step-by-step transition from monolith to microservices
- Avoids risky big-bang rewrites

2. System Stability

- Existing monolith continues to serve users
- No disruption to critical business workflows

3. Independent Scalability

- High-traffic or critical modules can be scaled independently
- Kubernetes supports auto-scaling where required

4. Faster Time to Market

- New features can be developed alongside migration
- Teams are not blocked by architectural changes

5. Reduced Operational Risk

- Issues in new microservices do not impact the entire system
- Easier rollback during failures

6. Better Resource Utilization

- Microservices consume resources only when needed
- Redis and Kafka reduce unnecessary load on core services

7. Improved Observability

- Centralized logging with Splunk
- Real-time monitoring and alerts via Datadog

8. Cloud Readiness

- Docker and Kubernetes make the system cloud-native
- AWS services enable elasticity and scalability

9. Performance Optimization

- Caching with Redis reduces database load
- Event-driven processing via Kafka improves responsiveness

10. Future Scalability

- Architecture is ready for full microservices adoption
- Database separation and service isolation can be added incrementally

- Disadvantages of the architecture

1. Increased Complexity

- Managing both monolith and microservices increases architectural complexity
- Requires careful coordination between components

2. Single Database Limitation

- Shared database creates tight coupling between services
- Limits independent schema evolution
- Becomes a scalability bottleneck

3. Operational Overhead

- Requires DevOps effort for Docker, Kubernetes, monitoring, and logging
- More moving parts to manage and maintain

4. Network Latency

- Inter-service communication introduces network calls
- Slower compared to in-process monolith calls

5. Data Consistency Challenges

- Harder to manage transactions across services
- Distributed transaction handling is complex

6. Debugging Difficulty

- Issues can span multiple services
- Requires strong logging, tracing, and correlation IDs

7. Partial Microservices Benefits

- Cannot fully leverage microservices advantages due to shared database
- True service isolation is not achieved yet

8. Learning Curve

- Teams need expertise in cloud, containers, and distributed systems
- Higher onboarding time for new developers

9. Cost Overhead

- Kubernetes, monitoring tools, and cloud resources increase infrastructure cost

10. Dependency Management

- Monolith remains a critical dependency
- Failure in the monolith can still impact the entire system

=====

3. API Gateway & Traffic Management

=====

Have you used an API Gateway?

Yes, we use a dedicated API Gateway service in our microservices architecture.

Instead of the monolith acting as the entry point, we implemented a separate API Gateway layer that serves as the single entry point for all client requests.

The API Gateway is responsible for:

- Routing requests to appropriate backend microservices
- Authentication and JWT validation
- Authorization enforcement
- Centralized logging and monitoring
- Request validation
- Rate limiting and throttling
- Handling cross-cutting concerns

This separation improves:

- Scalability
- Security
- Maintainability
- Clear separation of concerns

It also allows backend services to remain lightweight and focused only on business logic.

✓ How does the API Gateway manage load?

The API Gateway acts as a centralized traffic controller.

1 Load Balancing

- It routes incoming traffic to multiple instances of backend services.
 - In Kubernetes deployment, it works with Kubernetes Services and kube-proxy for load balancing.
 - Only healthy pods receive traffic (health checks enabled).
-

2 Rate Limiting

Rate limiting restricts how many requests a client can send within a specific time window.

Purpose:

- Prevent abuse
- Avoid DDoS-style traffic spikes
- Ensure fair usage among clients

Example:

- 100 requests per minute per user

If limit exceeds → 429 Too Many Requests

3 Throttling

Throttling controls request processing speed when traffic exceeds thresholds.

Instead of rejecting immediately:

- It slows down request processing
- Protects backend services
- Maintains system stability

Throttling is especially useful during sudden traffic spikes.

4 Auto-Scaling

Auto-scaling happens at the infrastructure level (Kubernetes).

- Horizontal Pod Autoscaler increases service instances
- Based on CPU, memory, or request count
- Scales down during low traffic to optimize cost

So load management is a combination of:

- API Gateway traffic control
- Kubernetes load balancing

Throttling and rate limiting are handled at the API Gateway level.

How is load balancing done in your project?

Architecture Short Note – Load Balancing, Service Registry & Routing (Kubernetes Deployment)

1 Who does Load Balancing?

In Kubernetes, load balancing is handled at multiple layers:

- **Kubernetes Service (ClusterIP / NodePort / LoadBalancer)**
 - Distributes traffic across healthy pods.
 - Uses kube-proxy (iptables/IPVS) internally.
 - Performs round-robin load balancing by default.
- **Ingress Controller (if used)**
 - Handles external HTTP/HTTPS traffic.
 - Routes traffic from outside cluster to internal services.
- **API Gateway (Application Level)**
 - Routes requests to correct backend services.
 - Applies rate limiting, throttling, authentication.
 - Relies on Kubernetes Service for actual pod-level balancing.

So:

External LB → Ingress → API Gateway → Kubernetes Service → Pods

2) Do we use Service Registry?

In Kubernetes, a separate service registry like Eureka is usually NOT required.

Reason:

- **Kubernetes has built-in service discovery.**
- **Each Service gets a DNS name.**
- **Pods communicate using:**

http://service-name:port

Kube-DNS / CoreDNS resolves service names to cluster IPs automatically.

So Kubernetes itself acts as:

- **Service Registry**
 - **Service Discovery mechanism**
-

3) How is Routing Done?

Routing happens in layers:

A) External Routing

Client → Load Balancer → Ingress Controller

B) Gateway Routing

API Gateway inspects:

- **Path (/audit/**)**
- **HTTP method**
- **Headers**
- **JWT claims**

Then forwards to appropriate internal service.

C) Internal Routing

API Gateway calls:

http://audit-service:8080

http://auth-service:8080

Kubernetes Service forwards request to one of the healthy pods.

Final Flow:

Client



Cloud Load Balancer



Ingress Controller



API Gateway



Kubernetes Service (DNS-based discovery)



Pod Instance (Load Balanced)

Key Points for Interview:

- **Kubernetes handles service discovery.**
- **kube-proxy handles pod-level load balancing.**
- **Ingress handles external routing.**
- **API Gateway handles application-level routing and traffic control.**
- **No separate Eureka needed in Kubernetes-native setup.**

=====

4. Database Design & Load Handling

=====

- How many databases do you have?

Currently, we use a single shared database across the application.

This is intentional because the system is in a transition phase from a monolithic architecture to microservices. Using a single database simplifies data consistency, reduces migration complexity, and ensures stability while services are being gradually extracted.

As part of the future roadmap, the plan is to move toward a database-per-service model once the microservices are fully isolated.

- How does your database handle load?

The database handles load through a combination of design optimizations and supporting infrastructure.

We use proper indexing on frequently queried columns to speed up read operations. Connection pooling is enabled to efficiently manage database connections and prevent connection exhaustion under high traffic.

To reduce direct database load, Redis caching is used for frequently accessed and read-heavy data. Pagination is applied for large result sets to avoid heavy queries.

Additionally, background processing and asynchronous workflows via Kafka help offload non-critical operations from synchronous database access, improving overall performance and scalability.

Connection Pooling

Connection pooling works by maintaining a pool of pre-created database connections that can be reused by multiple requests instead of creating a new connection each time.

When the application starts, the connection pool (for example, HikariCP in Spring Boot) creates a fixed number of database connections and keeps them ready.

When a request needs to access the database:

- It borrows an available connection from the pool
- Executes the query
- Returns the connection back to the pool instead of closing it

If all connections are in use, new requests wait until a connection becomes available, rather than creating unlimited connections and overwhelming the database.

This approach:

- Reduces connection creation overhead
- Improves performance under high traffic
- Prevents database connection exhaustion
- Ensures controlled and efficient use of database resources

=====

5. Caching Strategy

=====

- Is caching used in your project?

Yes, caching is used in our project.

We use Redis as a centralized in-memory cache to improve performance and reduce load on the database. Caching is mainly applied to frequently accessed, read-heavy data that does not change frequently.

- What is stored in the cache?

- When is data stored in cache?

- When and how is cache cleared?

=====

6. Logging & Monitoring

=====

- How is logging implemented?
- Which logging/monitoring tools are used?
- How is the logging tool integrated?
- How does application data reach the logging system?

=====

7. Authentication & Authorization

=====

- How is authentication managed?
- How is authorization managed?

=====

8. Inter-Service Communication

=====

- How do services communicate with each other?
- Synchronous vs asynchronous communication
- where is multithreading applied in your application ?

=====

9. Messaging / Event Streaming

=====

- Where is Kafka used in your project?
- Why is Kafka used?

=====

10. Background & Scheduled Jobs

=====

- Are there background jobs in your project?
- What kind of tasks run in the background?

=====

11. Bugs & Issue Handling

=====

- Bugs faced in the project
- How were those bugs identified?
- How were they resolved?

=====

12. Recent Contributions

=====

- Your recent contributions to the project

=====

13. Roles & Responsibilities

=====

- Your role in the team
- Your day-to-day responsibilities

=====

14. Challenges & Problem Solving

=====

- Difficulties faced in the project
- How did you tackle them?

=====

15. AOP (Aspect-Oriented Programming)

=====

- Where is AOP used in your project?
- Why was AOP used?

=====

Cloud

=====

- how do you manage load balancing ?

=====

Others

=====

1) HTTP vs HTTPS

Functional Info

We have an **internal audit application** that supports the **end-to-end audit lifecycle**.

Key modules:

- Dashboard
- Assessment area
- Project management
- Time tracking
- Audit report creation
- Recycle bin
- Audit history

Core audit objects:

- Risks and controls
- Strategic risks
- Objectives
- Workpapers

Major functionalities:

- Configurable **workflows**
- **Dynamic role- and permission-based access control**
- **Dynamic rule engine** for business logic
- **Workpaper upload and download**
- Customizable **naming and taxonomy**
- **SMTP-based email notifications**
- Automated **audit report generation**

Competitors::

AuditBoard

Workiva

Business user/admin flow

Admin Flow (Context Definition)

In the audit application, the **Admin** has full system-level access and configuration control.

Admin responsibilities include:

- Managing **taxonomy settings** across the application
- Defining **roles and permissions**
 - Creating new roles from existing system roles
 - Assigning permissions to roles
- **User management**
 - Creating users
 - Assigning roles to users
- Customizing the system
 - Renaming audit objects (risk, control, objective, workpaper, etc.)
 - Modifying taxonomy structures
- Configuring **SMTP settings** for email notifications

In addition to administrative capabilities, the **Admin can perform all operations available to a normal user**, including working on audits, assessments, workpapers, workflows, and reports.

User Flow (Context Definition)

A **User** has functional access based on the roles and permissions assigned to them.

User capabilities include:

- Accessing the **Assignment area**
 - Creating new assignments
 - Adding audit objects within an assignment
- Working with **Projects**
 - Linking a project to a specific audit area
 - Performing audits within that project
 - Adding predefined audit objects (risks, controls, objectives, workpapers, etc.)
- Managing audit objects
 - Creating, editing, and deleting objects
- Generating **audit reports** from assignment or project data
- Participating in **assessment workflows**
 - Being assigned to specific assessment phases
 - Receiving notifications based on assigned phases and tasks

User limitations:

- No access to **administrative areas** such as:
 - System settings
 - Statistics
 - Configuration modules

Application flow

TeamMate+ Audit – Overview

TeamMate+ Audit is a software application used by internal audit teams to **plan, execute, track, and report audits** efficiently.

It is widely used by organizations—especially in regulated industries like **banking**—to manage **risk, compliance, and governance** activities.

Who Uses TeamMate+ Audit?

1. Internal Audit Teams

- Conduct audits across departments
- Evaluate internal controls and processes
- Identify gaps, risks, and improvement areas

2. Risk Management Professionals

- Identify and assess operational, financial, and compliance risks
- Ensure risks are monitored and mitigated

3. Compliance Officers

- Ensure adherence to regulatory requirements and internal policies
- Track compliance issues and corrective actions

4. Audit Managers & Supervisors

- Plan audits and allocate resources
 - Monitor audit progress and quality
 - Review findings and final reports
-

Why Do Organizations Use TeamMate+ Audit?

Streamlined Audit Processes

- End-to-end audit lifecycle management
- Reduces manual work and spreadsheets

Enhanced Collaboration

- Centralized platform for auditors and stakeholders
- Clear visibility into tasks, findings, and status

Risk-Based Auditing

- Focuses audits on high-risk areas
- Improves early risk detection

Compliance Assurance

- Ensures audits align with regulatory standards
 - Maintains audit trails and documentation
-

Banking Sector Example (Simple Explanation)

Q1. Who Uses TeamMate+ Audit in a Bank?

- **Internal Audit Team**
Checks whether banking operations are safe, efficient, and compliant.
 - **Risk Management Team**
Identifies risks like fraud, credit risk, system failures, and operational issues.
 - **Compliance Officers**
Ensure adherence to RBI / central bank regulations and internal policies.
-

Q2. Why Do Banks Use TeamMate+ Audit?

Banks handle:

- Huge volumes of transactions
- Sensitive customer data
- Strict regulatory requirements

They must ensure:

- **Operational safety** – no fraud or transaction errors
- **Regulatory compliance** – RBI / central bank rules
- **Risk minimization** – early detection of issues
- **Accurate audit reporting** – for regulators and senior management

TeamMate+ Audit helps **automate, standardize, and control** these activities.

Q3. How Does a Bank Use TeamMate+ Audit?

1. Audit Planning

- Decide audit areas (loans, deposits, treasury, IT systems)
- Schedule audits and assign auditors
- Prioritize audits based on risk levels

2. Risk Assessment

- Identify high-risk areas:
 - Credit risk
 - Fraud risk
 - Operational risk
- Allocate more audit effort to critical areas

3. Audit Execution

- Auditors review transactions and controls
- Evidence and findings stored centrally
- Eliminates paperwork and scattered files

4. Reporting

- Automatically generates audit reports
- Clearly highlights:
 - Issues
 - Risks
 - Recommendations

5. Follow-Up (Cron-Based Tracking)

- Tracks whether audit issues are resolved
 - Sends reminders until actions are closed
 - Ensures accountability
-

Benefits to the Bank

- **Time Savings** – Less manual tracking and documentation
 - **Reduced Risk** – Systematic and consistent audits
 - **Improved Compliance** – Easier regulatory adherence
 - **Actionable Insights** – Identifies recurring issues and trends
 - **Transparency** – Clear visibility for management and regulators
-

Final Summary

In short:

In the banking sector, **TeamMate+ Audit** is used to **manage audits, control risks, ensure regulatory compliance, and improve audit efficiency**, helping banks remain **safe, compliant, and trustworthy**.

Technical Architecture

Tech stack used

Frontend:

- React 18, TypeScript, Vite
- Redux Toolkit

Backend:

- Java 17
- Spring Boot 3.x
- Spring Security, JWT, OAuth2
- RESTful APIs, Swagger (OpenAPI)
- JPA / Hibernate

Database:

- MySQL

Messaging & Cache:

- Kafka
- Redis

DevOps & Cloud:

- Git, GitHub
- Jenkins (CI/CD pipelines)
- Docker, Kubernetes
- AWS (EC2, S3, Lambda)

Testing & Quality:

- JUnit, Mockito
- Spring Boot Test
- SonarQube

Monitoring & Logging:

- Splunk
- Datadog

Tools:

- MobaXterm
- PuTTY

Design Pattern

Adapter design pattern::

Adapter Design Pattern – Application Flow Explanation

Our application follows an **adapted Adapter-based design pattern** to bridge HTTP requests with the service layer and enforce a structured business pipeline.

Why we use this pattern

- To **decouple controllers from business logic**
 - To enforce a **standard execution pipeline** for every request
 - To centralize **authorization, DB handling, and post-processing**
 - To make the system **extensible and maintainable**
-

Request Flow (End-to-End)

1. Controller Layer

- Receives the HTTP request
- Performs minimal request validation
- Creates an instance of the **Adapter**

2. Adapter Layer

- Acts as a bridge between the HTTP layer and business logic
- Translates request data into a business-friendly format
- Passes control to the **Execution Service**
- Ensures the controller remains thin

3. Execution Service (Business Pipeline)

- Executes a standardized pipeline:
 - User authentication & role validation

- Permission checks
- Connection handling
- Prepared statement creation
- Database operations
- Post-operation validations (audit logs, response checks, cleanup)

4. **Specific Business Service**

- Contains actual business logic
- Executes domain-specific rules
- Interacts with the database layer

5. **Database Layer**

- Performs CRUD operations using prepared statements
- Returns results to the service layer

6. **Post-Processing**

- Result validation
- Error handling
- Audit/logging (if applicable)
- Response preparation

7. **Response Flow**

- Travels back through Execution Service → Adapter → Controller
 - Controller sends the final HTTP response
-

High-Level Flow Summary

Controller



Adapter



Execution Service (Business Pipeline)



Specific Service



Database



Post-Operation Checks



Response back to Controller

Key Benefits

- **Loose coupling** between layers
 - **Single execution pipeline** for all business flows
 - **Easy to add new services** without touching controllers
 - **Cleaner controllers, reusable business logic**
 - **Consistent security and DB handling**
-
-

Example: Create Audit Assignment Flow

1 Controller (Thin – no business logic)

```
@RestController
@RequestMapping("/assignments")
public class AssignmentController {

    private final AssignmentAdapter assignmentAdapter;

    public AssignmentController(AssignmentAdapter assignmentAdapter)
    {
        this.assignmentAdapter = assignmentAdapter;
    }

    @PostMapping
    public ResponseEntity<?> createAssignment(@RequestBody
AssignmentRequest request) {
        return assignmentAdapter.createAssignment(request);
    }
}
```

👉 Controller only:

- Accepts HTTP request
- Delegates to Adapter
- No DB, no security, no logic

2 Adapter (HTTP → Business Bridge)

```
@Component
public class AssignmentAdapter {

    private final ExecutionService executionService;

    public AssignmentAdapter(ExecutionService executionService) {
        this.executionService = executionService;
    }
}
```

```

    public ResponseEntity<?> createAssignment(AssignmentRequest
request) {

        AssignmentContext context = new AssignmentContext();
        context.setRequest(request);

        ExecutionResult result = executionService.execute(
            context,
            BusinessOperation.CREATE_ASSIGNMENT
        );

        return ResponseEntity.status(result.getStatus())
            .body(result.getResponse());
    }
}

```

👉 Adapter responsibilities:

- Converts HTTP request to **business context**
- Calls execution pipeline
- Converts business response → HTTP response

3 Execution Service (Business Pipeline – Core)

This is where your **adapted design pattern** really lives.

```

@Service
public class ExecutionService {

    private final AuthService authService;
    private final BusinessServiceFactory serviceFactory;

    public ExecutionResult execute(BusinessContext context,
                                   BusinessOperation operation) {

        // 1. Authentication & Role validation
        authService.validateUser(context.getUser(), operation);
    }
}

```

```

        // 2. Get business service
        BusinessService service =
            serviceFactory.getService(operation);

        // 3. Execute business logic
        Object result = service.execute(context);

        // 4. Post-operation checks
        audit(context, result);

        return ExecutionResult.success(result);
    }

    private void audit(BusinessContext context, Object result) {
        // logging / audit trail / validation
    }
}

```

👉 Pipeline enforced here:

- Security
- Service routing
- Execution
- Post-processing

4 Business Service Interface (Template-like)

```

public interface BusinessService {
    Object execute(BusinessContext context);
}

```

5 Specific Business Service (Actual Logic)

```

@Service
public class CreateAssignmentService implements BusinessService {

    private final AssignmentRepository repository;
}

```



```

    public CreateAssignmentService(AssignmentRepository repository)
    {
        this.repository = repository;
    }

    @Override
    public Object execute(BusinessContext context) {

        AssignmentRequest req =
            ((AssignmentContext) context).getRequest();

        // Prepared statement via JPA / JDBC
        Assignment assignment = new Assignment();
        assignment.setName(req.getName());
        assignment.setCreatedBy(context.getUser().getId());

        return repository.save(assignment);
    }
}

```

👉 Only **business logic**, nothing else.

6 Factory (Routes Operation → Service)

```

@Component
public class BusinessServiceFactory {

    private final Map<BusinessOperation, BusinessService> services;

    public BusinessServiceFactory(List<BusinessService> serviceList)
    {
        services = Map.of(
            BusinessOperation.CREATE_ASSIGNMENT,
            serviceList.stream()
                .filter(s -> s instanceof CreateAssignmentService)
                .findFirst()
                .orElseThrow()
        );
    }
}

```

```
    public BusinessService getService(BusinessOperation operation) {  
        return services.get(operation);  
    }  
}
```

7 Flow Recap (Interview-friendly)

HTTP Request

↓

Controller (thin)

↓

Adapter (HTTP → Business)

↓

Execution Service (pipeline)

↓

Specific Business Service

↓

Database

↑

Post-operation checks

↑

HTTP Response

Patterns Used (Tell it straight)

- **Adapter Pattern** – Controller → Adapter → Service
- **Template Method (adapted)** – Fixed execution pipeline
- **Factory Pattern** – Resolve business service at runtime
- **Single Responsibility** – Each layer does one thing

Roles and responsibility

Roles and responsibility

=====

- Actively involved in development activities, with approximately 80% focus on new feature development and enhancements.
- Handle around 20% of sprint capacity for production support, client-reported defects, and bug fixes, ensuring minimal impact on business operations.
- Participate in technical and architectural discussions, contributing to design decisions, scalability considerations, and best practices.
- Actively participate in Agile Scrum ceremonies including:
 - Daily stand-ups
 - Sprint planning
 - Backlog grooming
 - PI increment planning
- Perform peer code reviews to ensure code quality, adherence to coding standards, performance optimization, and maintainability.

SDLC

Software Development Life Cycle (SDLC)

1. Feature discussion and high-level planning are carried out by higher management and product leadership.
2. Approved features are broken down into user stories (tickets) by the entire team, including developers and QA.
3. Steps 1 and 2 are part of PI (Program Increment) Planning, which is done for the upcoming 3 months of development and QA work.
4. Before the start of every sprint, stories are groomed, refined, estimated, and assigned to respective developers and QA engineers.
5. During sprint planning, stories are assigned to developers. Based on the number of stories (for example, 10 tickets) and available QA resources (for example, 2 QA), QA planning is done for functional and feature-wise testing.
6. Story estimation is done using story points, where:
 - 1 story point represents approximately 1 day (8 hours) of work
 - A story typically contains 2 to 3 story points
7. Once development for a story is completed, it is handed over to the QA team for functional and regression testing.
8. If QA identifies any issues, they are reported as internal bugs.
9. If a bug is related to a story developed by a specific developer, the bug is assigned back to the same developer for fixing.
10. After bug fixes, the QA team re-tests the functionality to ensure the issue is resolved and no regressions are introduced.
11. Once QA sign-off is completed, the feature is tested in the UAT environment by the Product Manager or business stakeholders.
12. As the PI approaches completion, a code freeze is applied on the staging branch.
13. After code freeze: (checked once)
 - Automation testers and developers run automated test suites
 - Developers execute unit and integration test cases
 - DevOps runs integration pipelines
 - Performance team conducts performance and load testing
14. Development activities continue on the main/UAT branch while the staging branch is used for testing and sanity validation.
15. After all validations are successful, the final build is released to production.

SDLC Flow for Client-Reported Bug

1. A bug is reported by the client and logged as a production issue or client defect in the tracking system.
2. The issue is analyzed by the team for impact, severity, and priority.
3. Based on the analysis, a dedicated team or developer is assigned to handle the bug.
4. The bug is planned and included in the next sprint under the allocated support capacity (typically 20% of the sprint).
5. The assigned developer works on the fix, following standard development practices including code changes and unit testing.
6. Once the fix is completed, the bug is moved to QA for validation and regression testing.
7. If QA finds any issues, the bug is reassigned to the developer for correction.
8. After successful QA validation, the fix is deployed to UAT for verification by the product manager or client (if required).
9. Once approved, the fix is merged into the staging branch and included in the next planned release.
10. After release, the fix is monitored in production to ensure the issue is fully resolved and no new issues are introduced.

Feature Deployment

RELEASE MANAGEMENT & DEPLOYMENT FLOW

1. FEATURE DEVELOPMENT PHASE

- Developers work on individual feature branches created from the main development branch.
- Each feature follows:
 - Code implementation
 - Unit testing
 - Local verification
 - Peer code reviews (PRs)
- All merges are done only after review approval and static checks.

2. CODE FREEZE

- Once all planned features for the release are completed, a code freeze is declared.
- During code freeze:
 - No new features are allowed.
 - Only critical bug fixes, security fixes, or release blockers are permitted.
 - Any fix requires explicit approval.
- This ensures stability and prevents scope creep before release.

3. STAGING BRANCH CREATION

- After code freeze, a staging branch is created from the stabilized development branch.
- The staging branch represents a production-like snapshot of the application.
- No direct feature development happens on this branch.

4. STAGING ENVIRONMENT DEPLOYMENT

- The staging branch is deployed to the staging environment.
- This environment closely mirrors production in terms of:
 - Infrastructure
 - Configuration
 - Data structure
 - Security settings

5. AUTOMATION & TEST EXECUTION

- Once deployed to staging, the CI/CD pipeline is triggered.
- The following automated tests are executed:
 - Unit tests
 - Integration tests
 - API contract tests
 - Regression test suites
 - Security and vulnerability scans (if applicable)
- Any test failure blocks the release and sends feedback to the team.

6. MANUAL & BUSINESS VALIDATION

- After automation passes, manual testing is performed:
 - Functional testing
 - Exploratory testing
 - User acceptance testing (UAT)

- Business stakeholders validate:
 - Feature correctness
 - Data integrity
 - End-to-end workflows

7. RELEASE BRANCH CREATION⁴⁵

- Once staging is fully validated, a release branch is created from the staging branch.
- This branch is strictly controlled and used only for:
 - Final bug fixes
 - Versioning
 - Release documentation
- No new features are allowed at this stage.

8. PRODUCTION DEPLOYMENT

- The release branch is deployed to the production environment via the CI/CD pipeline.
- Deployment strategies may include:
 - Blue-green deployment
 - Rolling deployment
 - Canary releases
- Health checks and smoke tests are executed post-deployment.

9. POST-DEPLOYMENT VERIFICATION

- After production deployment:
 - Application logs are monitored
 - Metrics and alerts are observed
 - Performance and error rates are validated
- Immediate rollback plans are kept ready in case of issues.

10. RELEASE CLOSURE

- Once production stability is confirmed:
 - The release branch is tagged
 - Changes are merged back to the main branch
 - Release notes are finalized
- The system is now ready for the next development cycle.

KEY BENEFITS OF THIS PROCESS:

- Controlled and predictable releases
- Early detection of defects
- Production stability
- Clear separation of development, validation, and release stages
- Reduced risk during deployment

Bugs and interview que

Bugs faced in the project

1) Latency bug

=====

LATENCY BUG – TIME TRACKING MODULE (Spring Boot + JDBC)

=====

1) ISSUE + EXISTING CODE

ISSUE:

- Time Tracking screen was slow
- ~5000 records loaded in single request
- High latency and poor user experience

ROOT CAUSE:

- JDBC used with custom DTO mapper
- Entire dataset processed in application layer

EXISTING CODE:

DAO LAYER:

```
public List<TimeEntryDTO> getAllTimeEntries() throws SQLException {
```

```
    String query = "SELECT t.id, t.hours, u.name, t.date " +  
        "FROM time_entry t " +  
        "JOIN user u ON t.user_id = u.id";
```

```
    Connection conn = dataSource.getConnection();  
    PreparedStatement ps = conn.prepareStatement(query);  
    ResultSet rs = ps.executeQuery();
```

```
    List<TimeEntryDTO> result = new ArrayList<>();
```

```
    // PROBLEM: Loop runs ~5000 times
```

```
    while (rs.next()) {  
        TimeEntryDTO dto = new TimeEntryDTO();
```

```
        dto.setId(rs.getLong("id"));  
        dto.setHours(rs.getDouble("hours"));  
        dto.setUserName(rs.getString("name"));  
        dto.setDate(rs.getDate("date").toLocalDate());
```

```
        result.add(dto);
```

```
    }
```

```
    return result;
}
```

PROBLEMS:

- Loads full dataset into memory
- 5000 DTO objects created
- High CPU usage due to loop
- Large response payload
- Application layer bottleneck

2) FIXED CODE + PERFORMANCE IMPROVEMENT

FIX 1: PAGINATION (MAIN FIX)

```
public List<TimeEntryDTO> getTimeEntries(int offset, int limit) throws SQLException {
```

```
    String query = "SELECT t.id, t.hours, u.name, t.date " +
        "FROM time_entry t " +
        "JOIN user u ON t.user_id = u.id " +
        "ORDER BY t.id " +
        "LIMIT ? OFFSET ?";
```

```
    Connection conn = dataSource.getConnection();
    PreparedStatement ps = conn.prepareStatement(query);
```

```
    ps.setInt(1, limit);
    ps.setInt(2, offset);
```

```
    ResultSet rs = ps.executeQuery();
```

```
    List<TimeEntryDTO> result = new ArrayList<>();
```

```
    // Now loop runs only 50–100 times
    while (rs.next()) {
        result.add(mapRow(rs));
    }
```

```
    return result;
}
```

FIX 2: OPTIMIZED MAPPER

```
private TimeEntryDTO mapRow(ResultSet rs) throws SQLException {
```

```

return TimeEntryDTO.builder()
    .id(rs.getLong("id"))
    .hours(rs.getDouble("hours"))
    .userName(rs.getString("name"))
    .date(rs.getDate("date").toLocalDate())
    .build();
}

```

FIX 3: CONTROLLER

```

@GetMapping("/time-entries")
public List<TimeEntryDTO> getTimeEntries(
    @RequestParam int page,
    @RequestParam int size) throws SQLException {

    int offset = page * size;
    return service.getTimeEntries(offset, size);
}

```

PERFORMANCE IMPROVEMENT:

BEFORE:

- 5000 records processed
- 5000 DTO objects
- High CPU + memory usage
- Large payload → slow API

AFTER:

- Only 50–100 records per request
- Reduced loop iterations
- Lower object creation
- Smaller payload

RESULT:

- Faster response time
- Reduced CPU usage
- Better scalability

3) WHY JDBC INSTEAD OF JPA/HIBERNATE

REASON 1: FULL CONTROL OVER SQL

- We write exact queries
- No ORM-generated queries
- Better control over joins and execution

REASON 2: PERFORMANCE PREDICTABILITY

- No Hibernate overhead:
 - No persistence context
 - No dirty checking
- Direct DB interaction

REASON 3: AVOID ORM ISSUES

- No N+1 problem
- No lazy/eager confusion
- Easier debugging

REASON 4: LEGACY SYSTEM

- Existing system already using JDBC
- Migration to JPA is risky and costly

REASON 5: BETTER FOR LARGE DATA

- Can control:
 - Pagination (LIMIT/OFFSET)
 - Fetch size
- Avoid loading huge datasets into memory

ONE LINE INTERVIEW ANSWER:

We used JDBC instead of JPA to have better control over SQL, avoid ORM overhead, and handle large datasets efficiently in a performance-critical legacy system.

“If we used JPA, the manual JDBC code like Connection, PreparedStatement, and ResultSet would be replaced by Spring Data JPA repositories. Pagination would be handled using Pageable instead of LIMIT/OFFSET, and entity mapping would be automatic instead of manual loops. However, the core issue would still remain if we fetched all records at once, so pagination is the real fix, not just switching from JDBC to JPA.”

=====

2)BUG SCENARIO: UI FIELD NOT RENDERING – DATA COMPATIBILITY ISSUE

ISSUE:

Users reported that a specific text field was not visible on the UI while loading a particular screen. The field existed in the layout configuration but was either blank or not rendered properly in the new UI.

IMPACT:

- Users were unable to view or edit a critical test-related field.
- Issue was intermittent and data-specific, making it difficult to reproduce.

- Affected only a subset of existing records.

INITIAL INVESTIGATION:

- Attempted to reproduce the issue locally and in lower environments.
- Verified:
 - UI configuration
 - Field visibility rules
 - Role-based permissions
 - API response payload
- The issue could NOT be reproduced consistently.
- New records were working fine, indicating the problem was not in the UI logic itself.

HYPOTHESIS:

- After detailed discussion and comparison between affected and non-affected records, we suspected a data-related issue rather than a UI or backend logic issue.

DATA ANALYSIS:

- Identified the database column backing the problematic text field.
- Executed queries to extract all distinct values from that column for affected records.
- While analyzing the data, we found:
 - Presence of restricted / non-printable / legacy special characters
 - Characters introduced from an older version of the UI
 - These characters were not compatible with the new UI rendering engine

ROOT CAUSE:

- The old UI allowed certain unsupported characters to be persisted in the database.
- The new UI framework had stricter rendering and validation rules.
- As a result, the field failed to render when encountering those invalid characters.

SOLUTION IMPLEMENTED:

1. DATA CLEANUP SCRIPT

- Wrote a backend SQL/data conversion script to sanitize the affected column.
- Removed or replaced unsupported and non-renderable characters.
- Ensured data consistency without impacting valid content.

2. PREVENTIVE VALIDATION

- Added validation at the backend layer to prevent such characters from being saved in the future.
- Ensured backward compatibility with the new UI.

RESULT:

- The affected field started rendering correctly for all users.
- No UI changes were required.
- Issue resolved permanently across environments.

KEY LEARNINGS:

- Legacy data can break modern UI frameworks.
- Not all UI issues are UI bugs — data integrity plays a critical role.

- Proper data validation and cleanup are essential during UI migrations.

3) Us military client facing document upload issue ,filter was changed

- How were those bugs identified?

- How were they resolved?

1. What was your contribution? -> RBAC

2. What was your biggest challenge? How do you overcome that?

DUPLICATE SURVEY RESPONSE ISSUE (CONCURRENCY PROBLEM)

SYSTEM DESIGN OVERVIEW

We have separate tables and services:

Survey
Questions
Survey Assignment
Survey Response

Relationships:

Admin creates a Survey
A Survey contains multiple Questions
Same Question can be reused across surveys
A Survey can be assigned to multiple Users
A User can fill multiple Surveys

DATABASE STRUCTURE

Table: survey_response

Columns:

id (Primary Key, auto-generated)

user_id

survey_id

question_id

response

Business Rule:

Each (user_id, survey_id, question_id) combination must be unique.

PROBLEM FACED

UI was sending duplicate requests due to server slowness

Backend used "check-then-insert" logic

Method was not synchronized

Failure Flow:

Thread 1 → Check → No record found

Thread 2 → Check → No record found

Thread 1 → Insert

Thread 2 → Insert

Result:

Duplicate rows created in survey_response table

WHY APPLICATION-LEVEL SOLUTIONS WERE NOT USED

Synchronization:

Reduces performance

Blocks threads

Not suitable for high concurrency

Frontend Debouncing:

Not reliable

Cannot control retries or race conditions

Check-Then-Insert:

Not atomic

Leads to race conditions

FINAL SOLUTION

Moved consistency enforcement to database layer

Added composite UNIQUE constraint:

```
ALTER TABLE survey_response  
ADD CONSTRAINT uk_user_survey_question  
UNIQUE (user_id, survey_id, question_id);
```

HOW IT SOLVES THE ISSUE

Thread 1 → Insert → Success

Thread 2 → Insert → Fails (unique constraint violation)

Result:

Only one record is stored

Duplicate entries prevented

Data integrity maintained

APPLICATION-LEVEL HANDLING

```
try {  
    surveyResponseRepository.save(entity);  
} catch (DataIntegrityViolationException ex) {  
    log.warn("Duplicate response detected");  
}
```

HANDLING EXISTING DUPLICATES

Step 1: Find duplicates

```
SELECT survey_id, question_id, user_id, COUNT() AS cnt  
FROM survey_response  
GROUP BY survey_id, question_id, user_id  
HAVING COUNT() > 1;
```

Step 2: Verify duplicates

```
SELECT t1.*  
FROM survey_response t1  
JOIN survey_response t2  
ON t1.survey_id = t2.survey_id  
AND t1.question_id = t2.question_id  
AND t1.user_id = t2.user_id  
AND t1.id > t2.id;
```

Step 3: Delete duplicates (keep lowest id)

```
DELETE t1
FROM survey_response t1
JOIN survey_response t2
ON t1.survey_id = t2.survey_id
AND t1.question_id = t2.question_id
AND t1.user_id = t2.user_id
AND t1.id > t2.id;
```

Step 4: Add unique constraint

```
ALTER TABLE survey_response
ADD CONSTRAINT unique_survey_question_user
UNIQUE (survey_id, question_id, user_id);
```

NOTE:

Take backup before deletion
Constraint will fail if duplicates exist
Follow steps in order
KEY DESIGN INSIGHT

Do not rely on application logic for concurrency control.
Always enforce critical constraints at the database level.

FINAL OUTCOME
Duplicate entries removed
System handles concurrency safely
No performance impact
Data integrity ensured
Issue permanently resolved
INTERVIEW ANSWER (SHORT)

We faced duplicate entries due to concurrent requests and non-atomic check-then-insert logic. Instead of using synchronization, we added a composite unique constraint at the database level, which ensured data consistency without impacting performance.

4. How did you optimise your API?

=====

HOW I OPTIMIZED OUR APIs

=====

API optimization was not done in one place.
It was done at multiple layers:

1) Database layer

- 2) Caching layer
- 3) Application layer
- 4) Authorization layer
- 5) Infrastructure layer

1) DATABASE OPTIMIZATION

Problem:

- Slow queries in audit reports
- N+1 query issues
- Large joins across audit, risk, findings tables

What I did:

- ✓ Added proper indexing
 - Composite indexes on (audit_id, status)
 - Indexes on foreign keys
 - Indexes on frequently filtered columns
- ✓ Solved N+1 issue
 - Replaced lazy loops with JOIN FETCH / custom queries
 - Used DTO projections instead of full entity load
- ✓ Reduced payload size
 - Used pagination (Pageable)
 - Limited columns using projections
- ✓ Optimized reporting queries
 - Used native queries for heavy aggregation
 - Avoided unnecessary entity hydration

Impact:

- Query time reduced significantly
- Lower DB CPU usage
- Faster report APIs

2) REDIS CACHING OPTIMIZATION

Problem:

Authorization lookup hitting DB every request.

Solution:

- ✓ Implemented cache-aside pattern
- ✓ Stored user permissions in Redis SET

- ✓ TTL strategy (15–30 mins)
- ✓ Explicit invalidation on role update

Impact:

- Reduced DB calls for auth by large margin
- Authorization became $O(1)$ lookup

③ APPLICATION LAYER OPTIMIZATION

- ✓ Removed unnecessary object mapping
 - Avoided deep entity graph serialization
- ✓ Used async processing for:
 - Email notifications
 - Report generation
 - Background tasks
- ✓ Used connection pooling (HikariCP tuning)
 - Optimized max pool size
 - Monitored slow queries
- ✓ Avoided excessive logging in production
 - Reduced I/O overhead

④ AUTHORIZATION OPTIMIZATION

Problem:

Heavy permission checks at service level.

Solution:

- ✓ Moved to AOP-based centralized permission check
- ✓ Avoided duplicate authorization logic in services
- ✓ Used Redis-based permission resolution

Impact:

- Cleaner code
- Faster permission validation
- Reduced redundant checks

⑤ INFRASTRUCTURE OPTIMIZATION

- ✓ Enabled API Gateway level rate limiting

- ✓ Enabled compression (GZIP)
- ✓ Proper thread pool configuration
- ✓ Monitored via logs & metrics
- ✓ Tuned Kubernetes resource limits

6 RESPONSE OPTIMIZATION

- ✓ Used pagination everywhere
- ✓ Avoided returning large nested JSON
- ✓ Used DTO instead of exposing entities
- ✓ Applied selective serialization

REAL PRODUCTION EXAMPLE

In reporting API:

Before:

- Multiple joins
- Full entity load
- No caching
- High response time

After:

- Optimized native query
- Added index
- Added pagination
- Moved heavy calculation async
- Cached static metadata

Result:

- ✓ API response time reduced
- ✓ DB load reduced
- ✓ No timeout in peak usage

SHORT INTERVIEW VERSION

"I optimized APIs at multiple layers — database indexing, query tuning, fixing N+1 issues, Redis caching for authorization, DTO projections to reduce payload size, async processing for heavy tasks, and infrastructure tuning. This significantly reduced response time and DB load while improving scalability."

5. Which endpoint/API you created?

DEPLOYED ROLE & PERMISSION MANAGEMENT APIs

We implemented dedicated APIs to manage dynamic roles and permissions in the system. These APIs are not simple CRUD endpoints — they include internal validation, mapping logic, cache handling, and security enforcement.

1 API :: getRoleList

Endpoint:

GET /api/roles

Purpose:

Fetch all available roles (System + Custom).

Internal Logic:

- Fetch roles from database
- Identify system roles (non-editable flag)
- Mark custom roles (editable)
- Apply organization-level filtering if applicable
- Return structured role response

Response Example:

```
[
  {
    "roleId": 1,
    "roleName": "ADMIN",
    "type": "SYSTEM",
    "editable": false
  },
  {
    "roleId": 7,
    "roleName": "CUSTOM_AUDITOR",
    "type": "CUSTOM",
    "editable": true
  }
]
```

Additional Handling:

- Authorization check before returning roles
- Redis caching (optional for optimization)

② API :: getPermissionListForRole

Triggered When:

User clicks on a specific role in UI.

Endpoint:

GET /api/roles/{roleId}/permissions

Purpose:

Load complete permission matrix for the selected role.

Internal Logic:

- Validate role existence
- Fetch role-permission mappings
- Group permissions by module (Survey, Audit, User Mgmt, etc.)
- Mark selected vs unselected permissions
- Handle system role restrictions (view-only if system role)

Response Example:

```
{
  "roleId": 7,
  "roleName": "CUSTOM_AUDITOR",
  "permissions": [
    {
      "module": "SURVEY",
      "permissionCode": "SURVEY_CREATE",
      "assigned": true
    },
    {
      "module": "SURVEY",
      "permissionCode": "SURVEY_DELETE",
      "assigned": false
    }
  ]
}
```

Security Handling:

- Permission to view role configuration required
- Dynamic role-permission resolution via Redis → DB fallback

③ API :: updateRolesToPermission

Triggered When:

User edits permissions and clicks SAVE.

Endpoint:

PUT /api/roles/{roleId}/permissions

Request Body:

```
{
  "permissionIds": [101, 102, 205, 309]
}
```

Purpose:

Update role-to-permission mapping.

Internal Execution Flow:

Step 1:

- Validate role exists
- Check if role is editable
- Reject update if system role

Step 2:

- Fetch existing permissions
- Calculate:
 - Permissions to add
 - Permissions to remove

Step 3:

- Perform batch insert/delete in mapping table
- Maintain transactional integrity

Step 4:

- Invalidate Redis cache:
 - ROLE_PERMISSIONS:{roleId}
 - USER_ROLES cache for impacted users

Step 5:

- Optional audit **logging** of changes

Response:

```
{
  "status": "SUCCESS",
  "message": "Role permissions updated successfully"
}
```



Why This Design Matters

-
- ✓ Clear separation between role and permission management
 - ✓ System roles protected (non-editable)
 - ✓ Custom roles flexible and configurable
 - ✓ Redis cache keeps authorization fast
 - ✓ Cache invalidation ensures immediate reflection
 - ✓ Transaction-safe updates
 - ✓ Fully dynamic RBAC

FLOW SUMMARY

UI → getRoleList
→ Click Role → getPermissionListForRole
→ Edit → updateRolesToPermission
→ Cache invalidation
→ Dynamic authorization reflects immediately

=====

These APIs collectively power the dynamic RBAC system
deployed in production.

=====

6. What was the DB schemas you worked on?

**I primarily worked on RBAC (Role-Based Access Control) schemas
and core business schemas for an enterprise audit application.**

A) RBAC SCHEMAS (Most Critical Area)

1) users

-
- id (UUID, Primary Key)
 - username (Unique)
 - email (Unique)
 - password_hash
 - status
 - created_at
 - updated_at

Note:

UUID used because:

- Distributed microservices environment

- Avoid sequential ID exposure
- Better merge compatibility across services

2) roles

-
- id (UUID, Primary Key)
 - role_name (Unique)
 - description
 - role_type (GLOBAL / ASSESSMENT)
 - created_at
 - updated_at

3) permissions

-
- id (UUID, Primary Key)
 - permission_key (Unique)
 - module_name
 - action_type (READ / WRITE / DELETE / APPROVE)
 - created_at
 - updated_at

4) user_roles (Mapping Table)

-
- id (UUID, Primary Key)
 - user_id (UUID, FK → users.id)
 - role_id (UUID, FK → roles.id)
 - assessment_id (UUID, Nullable, for context-based role override)
 - assigned_at
 - assigned_by

5) role_permissions (Mapping Table)

-
- id (UUID, Primary Key)
 - role_id (UUID, FK → roles.id)
 - permission_id (UUID, FK → permissions.id)
 - created_at

Why UUID here?

- Security-sensitive module
- Prevent predictable ID enumeration
- Used across microservices
- Compatible with Redis caching keys

B) SURVEY / RESPONSE SCHEMA

6) survey

-
- id (UUID, Primary Key)
 - title
 - description
 - created_by (UUID)
 - created_at
 - updated_at

7) question

-
- id (UUID, Primary Key)
 - question_text
 - question_type
 - is_mandatory
 - created_at

8) survey_assignment

-
- id (UUID, Primary Key)
 - survey_id (UUID, FK)
 - user_id (UUID, FK)
 - assigned_at
 - due_date
 - status

9) survey_response

-
- id (UUID, Primary Key)
 - user_id (UUID, FK)
 - survey_id (UUID, FK)
 - question_id (UUID, FK)
 - response
 - submitted_at

Composite Unique Constraint:

UNIQUE (user_id, survey_id, question_id)

Purpose:

Prevent duplicate entries under concurrent API calls.

C) GENERIC CORE BUSINESS SCHEMAS

10) audit

- id (UUID, Primary Key)
- audit_name
- audit_type
- status
- start_date
- end_date
- created_by (UUID)

11) findings

- id (UUID, Primary Key)
- audit_id (UUID, FK)
- severity
- description
- status
- owner_id (UUID)
- created_at

12) documents

- id (UUID, Primary Key)
- reference_id (UUID)
- reference_type
- s3_path
- uploaded_by (UUID)
- uploaded_at

13) notifications

- id (UUID, Primary Key)
 - user_id (UUID)
 - message
 - is_read
 - created_at
-

Why UUID Instead of Auto-Increment?

- Microservices architecture
- Avoid collision during data migration
- Prevent ID guessing
- Easier replication & sharding
- Safe for distributed systems

Summary (Interview Ready)

I worked extensively on:

- 1) RBAC schemas (users, roles, permissions, mappings)
- 2) Context-based authorization schemas
- 3) Survey & response management schemas
- 4) Audit core business schemas
- 5) Document & notification schemas

Most tables used UUID as primary key
to support distributed architecture, security, and scalability.

7. How implementing Redis & Kafka improved response time?

8. How is performance testing done in your project?

9. How do you know response code improved by x second?

10. SDLC of your project

11. Tell when your responsibility over feature you delivered

12. How do you know when Redis cache should flush out the data & where do you write the logic?

13. do you know multithreading and how you implemented in your project

14. not reproducible issue?

Yes — this duplicate survey response issue was not consistently reproducible.

How We Tried to Recreate It:

- 1) Sent multiple rapid requests from frontend.
- 2) Used Fiddler to fire parallel API requests.
- 3) Enabled Chrome DevTools network throttling (3G mode).
- 4) Tested with concurrent sessions.

Still not reproducible consistently.

Observation:

- It was a race condition.

- Timing dependent.
- Occurred only under specific production latency scenarios.
- Non-deterministic in local/dev environments.

Resolution Approach:

Instead of chasing reproduction,
we focused on structural prevention.

Upgraded schema and enforced:

UNIQUE (user_id, survey_id, question_id)

Now:

- Even if any client sends duplicate concurrent requests,
- Even if retries happen due to network,
- Even if new frontend is integrated,

Database prevents duplicate entries.

Key Learning:

- Some concurrency bugs are rare and timing-sensitive.
- Do not rely only on application-level fixes.
- Enforce critical business rules at database level.
- Fix the architecture, not just the symptom.

16. Why do we need integer pk if we have uuid?

=> UUID ensures global uniqueness and prevents ID enumeration. However, BIGINT primary keys provide better indexing, faster joins, lower storage cost, and improved write performance due to sequential inserts. So we use BIGINT internally for database efficiency and UUID externally for security and distributed system compatibility.

Authentication

Authentication in our project is implemented using JWT (JSON Web Token) and follows a stateless authentication model.

The application is stateless, and authentication is handled at the API Gateway level. Spring Security is used to validate and authenticate requests using a chain of security filters.

When a user logs in:

- If no JWT is present, the API Gateway validates the user credentials (user ID and password) against the database.
- Once the credentials are successfully verified, the system generates a JWT.

The generated JWT contains:

- User ID
- User roles
- Token metadata (expiry, issuer, etc.)

Two types of tokens are issued:

1. Access Token
2. Refresh Token

The Access Token:

- Is short-lived
- Is stored in the frontend Redux store
- Is sent with every API request in the Authorization header
- Is used to authenticate and authorize the user for each request

The Refresh Token:

- Is long-lived
- Is stored securely in browser cookies
- Is used only when the access token expires or becomes invalid

For every incoming request:

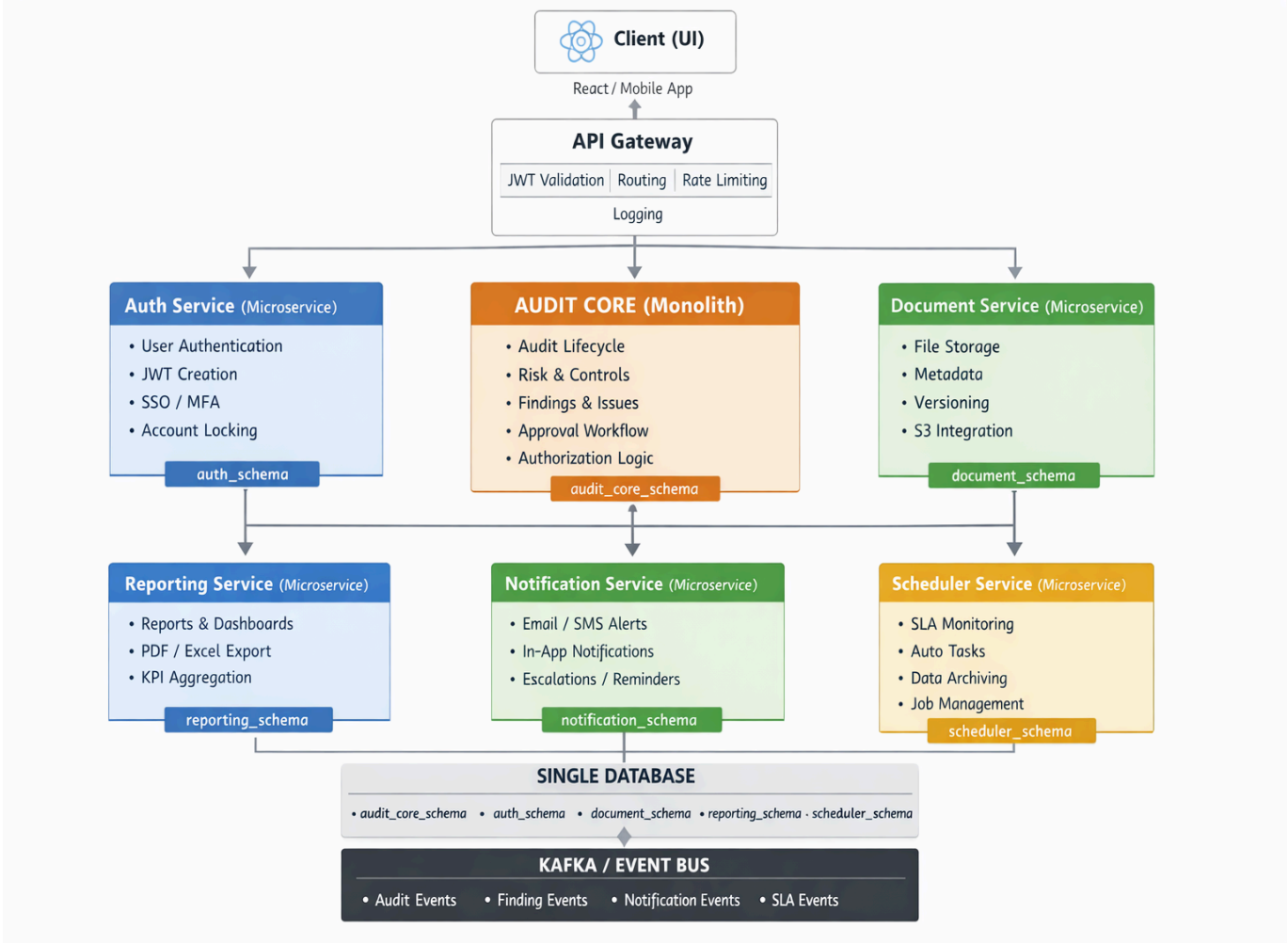
- The API Gateway validates the access token using Spring Security filters
- If the token is missing, expired, the request is rejected

If the access token is expired:

- The refresh token is sent to generate a new access token
- If the refresh token is also invalid, the user is logged out and access is denied

Overall, authentication is enforced through a combination of JWT-based stateless security and Spring Security filter chains.

Diagram



API created

Authorisation

Authorisation

Previous rbac

```
=====
PREVIOUS AUTHORIZATION SETUP
(Static / Hard-coded Roles)
=====
```

1) PREDEFINED ROLES IN THE SYSTEM

The application had predefined roles already configured in the backend code.
These roles were fixed and could NOT be modified by the admin.

Example predefined roles:

ADMIN
AUDITOR
REVIEWER
OBSERVER

Internally Spring Security stores roles with prefix ROLE_ :

ROLE_ADMIN
ROLE_AUDITOR
ROLE_REVIEWER
ROLE_OBSERVER

2) ADMIN CREATES USER AND ASSIGNS ROLE

Admin could create a new user and assign one of the predefined roles.

Example:

User	Assigned Role
Tejas	ADMIN
Rahul	AUDITOR
Priya	REVIEWER

Example DB record:

username : rahul
role : ROLE_AUDITOR

3) PROTECTING APIs USING @PreAuthorize

To restrict access to APIs we used Spring Security annotation @PreAuthorize.

Example:

```
@PreAuthorize("hasRole('ADMIN')")  
public void deleteAudit(Long id){  
    auditRepository.deleteById(id);  
}
```

Meaning:

Only users with ADMIN role can execute this method.

4) HOW @PreAuthorize CHECKS ROLE

When request comes:

```
Client Request  
↓  
JWT Token Validated  
↓  
Roles extracted from token  
↓  
Authentication object created  
↓  
Stored inside SecurityContext
```

Example SecurityContext:

User : rahul
Authorities : [ROLE_AUDITOR]

Now Spring evaluates:

```
@PreAuthorize("hasRole('ADMIN')")
```

Spring internally converts this to:

ROLE_ADMIN

Then it checks:

Does ROLE_ADMIN exist in authentication authorities?

5) FINAL AUTHORIZATION DECISION

CASE 1 : ROLE PRESENT

Authorities : [ROLE_ADMIN]

Required : ROLE_ADMIN

Result → Access Granted

Method executes

CASE 2 : ROLE NOT PRESENT

Authorities : [ROLE_AUDITOR]

Required : ROLE_ADMIN

Result → AccessDeniedException

HTTP 403 Forbidden

Method execution blocked

SHORT SUMMARY

- Roles were predefined in the system
- Admin could only assign roles to users
- APIs were protected using @PreAuthorize
- Spring fetched roles from SecurityContext
- If role matched → method executed
- If role missing → access denied

RBAC AUTHORIZATION FLOW – PROJECT NOTES

(Multi-Tenant + Assessment Based Access)

SYSTEM IDENTIFIERS

tenantId → identifies client organization

userId (UUID) → identifies user

assessmentId → identifies audit/assessment context

Authorization decision is based on:

tenantId + userId + assessmentId

=====

=====

A) ADMIN OPERATIONS

1. ADMIN CREATES USER AND ASSIGNS ROLE

Admin creates a user from the UI and assigns role per assessment.

Example UI action:

User Email: tejas@gmail.com

Tenant: ABC Bank

Assessment: A101

Role: AUDITOR

DATABASE TABLES USED

TABLE: users

user_id (UUID)
tenant_id
email
name
status

Example:

user_id = U123
tenant_id = T100
email = tejas@gmail.com

TABLE: assessments

assessment_id
tenant_id
assessment_name

Example:

assessment_id = A101

TABLE: user_assessment_roles

Stores role of a user for a specific assessment.

id
tenant_id
user_id
assessment_id
role_id
assigned_at

Example record:

tenant_id T100
user_id U123
assessment_id A101
role_id AUDITOR

Meaning:

User U123 is AUDITOR in assessment A101

2. ROLE → PERMISSION MAPPING

Roles and permissions are stored separately.

TABLE: roles

role_id
role_name

Example:

ADMIN
AUDITOR
REVIEWER

TABLE: permissions

permission_id
permission_key

Example:

AUDIT_CREATE
AUDIT_EDIT
WORKPAPER_UPLOAD
REPORT_EXPORT

TABLE: role_permissions

role_id
permission_id

Example records:

AUDITOR → AUDIT_CREATE
AUDITOR → AUDIT_EDIT
AUDITOR → WORKPAPER_UPLOAD

REVIEWER → AUDIT_REVIEW
REVIEWER → WORKPAPER_APPROVE

ADMIN → all permissions

=====

B) USER REQUEST FLOW

1. HOW REQUEST URL LOOKS

Example request:

POST /api/v1/assessments/A101/workpapers

Headers:

Authorization: Bearer JWT_TOKEN

From URL we extract:

assessmentId = A101

2. JWT PAYLOAD CONTENT

JWT contains identity information only.

Example payload:

```
{
  "tenantId": "T100",
  "userId": "U123",
  "email": "tejas@gmail.com", or we can send uuid
  "exp": 1712345678
}
```

Important fields used:

tenantId
userId

These values are extracted during authentication.

3. CUSTOM AUTHORIZATION ANNOTATION

APIs are secured using a custom annotation.

Example:

```
@RequirePermission("WORKPAPER_UPLOAD")
```

API example:

```
@RequirePermission("WORKPAPER_UPLOAD")  
@PostMapping("/assessments/{assessmentId}/workpapers")  
public ResponseEntity uploadWorkpaper() {  
    ...  
}
```

Meaning:

Only users having permission WORKPAPER_UPLOAD
for that assessment can execute this API.

4. AOP AUTHORIZATION ASPECT

The aspect intercepts the API and checks permission.

```
@Aspect  
@Component  
public class AuthorizationAspect {  
  
    @Autowired  
    private RedisService redisService;  
  
    @Autowired  
    private RoleService roleService;
```

```

@Before("@annotation(requirePermission)")
public void checkPermission(RequirePermission requirePermission) {

    String requiredPermission = requirePermission.value();

    Authentication auth =
        SecurityContextHolder.getContext().getAuthentication();

    String userId = auth.getName();

    HttpServletRequest request =
        ((ServletRequestAttributes) RequestContextHolder
            .getRequestAttributes()).getRequest();

    String tenantId = request.getHeader("tenantId");
    String assessmentId = request.getParameter("assessmentId");

    String role = redisService.getUserRole(tenantId, userId, assessmentId);

    Set<String> permissions = redisService.getPermissionsForRole(role);

    if(!permissions.contains(requiredPermission)){
        throw new AccessDeniedException("Permission denied");
    }
}
}

```

5. REDIS CONFIGURATION AND METHODS

Redis is used as a caching layer for RBAC.

Main Redis Keys:

User role per assessment:

role:user:{tenantId}:{userId}:{assessmentId}

Example:

role:user:T100:U123:A101 → AUDITOR

Role permissions:

perm:role:{roleName}

Example:

perm:role:AUDITOR

Value stored as SET:

AUDIT_CREATE
AUDIT_EDIT
WORKPAPER_UPLOAD

Redis Service Example:

```
public String getUserRole(String tenantId,String userId,String assessmentId){  
  
    String key = "role:user:" + tenantId + ":" + userId + ":" + assessmentId;  
  
    return redisTemplate.opsForValue().get(key);  
}
```

```
public Set<String> getPermissionsForRole(String role){  
  
    String key = "perm:role:" + role;  
  
    return redisTemplate.opsForSet().members(key);  
}
```

6. CACHE INSERTION WITH TTL

When cache miss occurs we fetch from DB and store in Redis.

```
public void cacheUserRole(String tenantId,String userId,String assessmentId,String role){  
  
    String key = "role:user:" + tenantId + ":" + userId + ":" + assessmentId;  
  
    redisTemplate.opsForValue().set(key, role, 15, TimeUnit.MINUTES);  
}
```

Role permissions cache:

```
public void cacheRolePermissions(String role, Set<String> permissions){  
  
    String key = "perm:role:" + role;  
  
    redisTemplate.opsForSet().add(key, permissions.toArray(new String[0]));  
  
    redisTemplate.expire(key, 30, TimeUnit.MINUTES);  
}
```

7. GRANULAR CACHE EVICTION

When admin updates role or permission we delete only specific keys.

Example role change:

```
redisTemplate.delete("perm:role:AUDITOR");
```

Example user role update:

```
redisTemplate.delete("role:user:T100:U123:A101");
```

The next request will repopulate the cache.

8. MEMORY BASED EVICTION POLICY

Redis configuration:

```
maxmemory 4gb  
maxmemory-policy allkeys-lfu
```

Meaning:

Redis removes least frequently used keys when memory is full.

9. IMPORTANT THINGS TO FOCUS

-
- Use userId(UUID) instead of email for Redis keys
 - Avoid storing roles inside JWT
 - Always invalidate cache after role updates
 - Use Redis SET for permission lookup
 - Use cache-aside pattern for loading RBAC data
 - Never use global FLUSHALL in production
 - Ensure Redis TTL prevents stale permissions

=====

FINAL AUTHORIZATION FLOW

User Request
↓
JWT validated
↓
Extract tenantId + userId
↓
Extract assessmentId from API
↓
Redis lookup → role:user:{tenantId}:{userId}:{assessmentId}
↓
Role obtained
↓
Redis lookup → perm:role:{roleName}
↓
Permission list obtained
↓
Aspect checks required permission
↓
Allow request OR throw AccessDeniedException

=====

END

@Async

=====

@Async — History Service Summary

=====

1) Purpose in History Service

We use @Async in History Service to:

- Decouple audit/history persistence from main business flow
- Reduce API response time
- Prevent history failure from breaking main request
- Achieve eventual consistency

History is treated as non-blocking, secondary operation.

2) Basic Implementation

@EnableAsync // required at configuration level at main class

```
@Service
public class HistoryService {

    @Async("historyExecutor")
    public CompletableFuture<Void> saveAsync(HistoryEvent event) {
        try {
            historyRepository.save(event);
        } catch (Exception ex) {
            log.error("History persistence failed for {}", event.getId(), ex);
        }
        return CompletableFuture.completedFuture(null);
    }
}
```

3) How It Works Internally

```
Caller Thread
    ↓
Spring AOP Proxy
    ↓
TaskExecutor.submit()
    ↓
Separate background thread executes saveAsync()
```

Key Points:

- Runs in different thread
- Separate transaction context

- Exceptions do NOT propagate to caller
- Main request returns immediately

4) Ensuring Main Request Does Not Fail

- Async method runs in separate thread
- Exceptions are caught inside method
- No exception bubbles back to controller
- History failure does not affect API response

5) Critical Data Consistency Requirement

History must run **ONLY** after main transaction commits.

Correct pattern:

```
@TransactionalEventListener(phase = TransactionPhase.AFTER_COMMIT)
public void handle(TaskUpdatedEvent event) {
    historyService.saveAsync(event);
}
```

This ensures:

- No history written for rolled-back transactions

6) Retry for Transient Failures

```
@Async("historyExecutor")
@Retryable(
    value = { Exception.class },
    maxAttempts = 3,
    backoff = @Backoff(delay = 2000)
)
public CompletableFuture<Void> saveAsync(HistoryEvent event) {
    historyRepository.save(event);
    return CompletableFuture.completedFuture(null);
}
```

Solves:

- Temporary DB failure
- Connection pool hiccups
- Deadlocks

7) Thread Pool Configuration

```
-----  
  
@Bean(name = "historyExecutor")  
public Executor historyExecutor() {  
    ThreadPoolTaskExecutor executor = new ThreadPoolTaskExecutor();  
    executor.setCorePoolSize(5);  
    executor.setMaxPoolSize(10);  
    executor.setQueueCapacity(500);  
    executor.initialize();  
    return executor;  
}
```

Prevents:

- Blocking request threads
- Silent task dropping
- Thread exhaustion

8) Limitations of @Async for History

- In-memory execution (lost if app crashes)
- No durability guarantee
- No built-in retry unless added
- No delivery tracking
- No exactly-once guarantee

9) When @Async Is Acceptable for History

- Audit is non-critical
- Minor data loss acceptable
- Low-to-moderate traffic
- Eventual consistency allowed

10) When @Async Is NOT Enough

If history is:

- Compliance-critical
- Financially regulated
- Requires guaranteed persistence
- Needs high throughput

Better approach:

- Outbox Pattern
- Kafka-based event-driven architecture

11) Interview-Ready Summary

In our system, History Service was executed asynchronously to prevent audit logging from increasing API latency. The async method runs in a dedicated thread pool and is triggered only after the main transaction commits using `@TransactionalEventListener(AFTER_COMMIT)`. Failures are logged and retried when necessary. Since `@Async` is thread-based and not crash-safe, stronger durability guarantees would require an event-driven approach such as Kafka or the Outbox pattern.

=====

END

=====

StatusCodeExceptionHandling

=====

Exception Handling Using @ResponseStatus (Handling Specific HTTP Status Codes)

=====

1) Purpose

@ResponseStatus is used to map a specific exception to a specific HTTP status code.

It allows:

- Clean separation of business logic and HTTP response logic
- Automatic HTTP status mapping
- Avoiding manual ResponseEntity creation everywhere

2) Basic Usage on Custom Exception

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class HistoryNotFoundException extends RuntimeException {
    public HistoryNotFoundException(String message) {
        super(message);
    }
}
```

When this exception is thrown:

```
throw new HistoryNotFoundException("History not found");
```

Spring automatically returns:

HTTP 404 Not Found

3) With Custom Reason

```
@ResponseStatus(
    value = HttpStatus.BAD_REQUEST,
    reason = "Invalid history request"
)
public class InvalidHistoryRequestException extends RuntimeException {
}
```

Response:

HTTP 400 Bad Request

Body:

"Invalid history request"

Note:

Using "reason" produces default error response body.

4) Using @ExceptionHandler (More Control)

For structured JSON responses, prefer global exception handling:

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(HistoryNotFoundException.class)
    @ResponseStatus(HttpStatus.NOT_FOUND)
    public ErrorResponse handleHistoryNotFound(
        HistoryNotFoundException ex) {

        return new ErrorResponse(
            404,
            ex.getMessage(),
            LocalDateTime.now()
        );
    }
}
```

This gives:

- Custom JSON body
- Control over structure
- Consistent API contract

5) Handling Multiple Status Codes

Different exceptions → different status codes:

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class HistoryNotFoundException extends RuntimeException {}

@ResponseStatus(HttpStatus.BAD_REQUEST)
public class InvalidHistoryRequestException extends RuntimeException {}

@ResponseStatus(HttpStatus.CONFLICT)
public class DuplicateHistoryException extends RuntimeException {}
```

6) When Exception Is Thrown in Service

```
@Service
public class HistoryService {

    public History getHistory(Long id) {
        return historyRepository.findById(id)
            .orElseThrow(() ->
                new HistoryNotFoundException("History not found"));
    }
}
```

Spring automatically converts exception → HTTP status.

7) Difference: @ResponseStatus vs ResponseEntity

@ResponseStatus:

- Static mapping
- Cleaner
- Less boilerplate
- Good for simple APIs

ResponseEntity:

- Dynamic status
- Full response control
- Used when status depends on runtime condition

8) Limitations of @ResponseStatus

- Fixed HTTP status
- Harder to return dynamic error body
- Not ideal for complex error responses
- Does not handle validation errors automatically

9) Best Practice in Production

For structured APIs:

Use:

- @RestControllerAdvice
- @ExceptionHandler
- Custom error response object

Avoid using "reason" in @ResponseStatus for REST APIs

because it limits response body customization.

10) Interview-Ready Summary

@ResponseStatus allows mapping custom exceptions to specific HTTP status codes declaratively. When the exception is thrown, Spring automatically converts it to the defined HTTP status. For more structured error handling and custom JSON responses, it is recommended to use @RestControllerAdvice with @ExceptionHandler instead of relying solely on @ResponseStatus.

=====
END
=====

===== How Exceptions Are Thrown & Detected in Spring Boot =====

1) Do We Need to Explicitly Throw Exceptions?

Yes — for custom business exceptions, you must explicitly throw them.

Example:

```
if (history == null) {  
    throw new HistoryNotFoundException("History not found");  
}
```

Spring does NOT automatically throw your custom exceptions.
You define when business logic is violated.

2) Where Can Exceptions Be Thrown?

Exceptions can originate from:

- Controller layer
- Service layer
- Repository layer
- Validation layer
- Framework itself (Spring, Hibernate, Jackson, etc.)

Spring detects any unhandled exception that propagates up to the DispatcherServlet.

3) How Spring Detects Exceptions

Flow:

Controller → Service → Repository



Exception thrown



Bubbles up call stack



Spring DispatcherServlet catches it



Checks:

- @ExceptionHandler
- @RestControllerAdvice
- @ResponseStatus
- Default Exception Resolver



Converts to HTTP response

4) Example: Service Throws Exception

@Service

public class HistoryService {

 public History getHistory(Long id) {

 return historyRepository.findById(id)

 .orElseThrow(() ->

 new HistoryNotFoundException("History not found"));

 }

}

If not caught manually,

Spring automatically routes it to exception handlers.

5) Framework-Thrown Exceptions (Automatic)

Some exceptions are thrown automatically by Spring:

- MethodArgumentNotValidException (validation failure)
- HttpMessageNotReadableException (invalid JSON)
- DataIntegrityViolationException (DB constraint violation)
- MissingServletRequestParameterException

You do NOT throw these manually.

Spring throws them internally.

6) Can Spring Detect Exceptions From Anywhere?

Spring only handles exceptions that:

- Occur inside Spring-managed beans
- Happen during HTTP request processing
- Propagate to the DispatcherServlet

It will NOT detect:

- Exceptions caught and swallowed manually
- Exceptions inside separate threads unless handled properly
- Exceptions outside request lifecycle

7) What Happens If You Catch It?

```
try {  
    historyRepository.save(entity);  
} catch (Exception e) {  
    log.error("Failed");  
}
```

If you catch and do NOT rethrow,
Spring never sees it.

To let Spring handle it:

```
catch (Exception e) {  
    throw new HistorySaveException("Failed to save history");  
}
```

8) Async Case (Important)

For @Async methods:

- Exception does NOT propagate to controller thread.
- Must handle inside async method.
- Or use CompletableFuture and handle via exceptionally().

9) Summary

- Business exceptions must be explicitly thrown.
- Spring automatically catches uncaught exceptions.
- Exceptions bubble up the call stack.

- @ExceptionHandler / @ResponseStatus decide HTTP response.
- If you catch and suppress exception, Spring cannot detect it.
- Async exceptions do not propagate to request thread.

```
=====
END
=====
```


Deployment Strategy

Rolling Update Deployment Strategy

1) Purpose

Rolling Update is a deployment strategy where:

- New version instances are deployed gradually
- Old version instances are terminated progressively
- System remains available during deployment
- No full downtime occurs

Used in:

- Microservices
- Kubernetes-based deployments
- High-availability systems

2) How It Works

Instead of replacing all instances at once:

Old Version (v1):

Instance 1
Instance 2
Instance 3
Instance 4

Step-by-step process:

Step 1:

Terminate Instance 1 (v1)
Start Instance 1 (v2)

Step 2:

Terminate Instance 2 (v1)
Start Instance 2 (v2)

Continue until all are upgraded.

Traffic always flows to healthy instances.

3) Example in Kubernetes

spec:

replicas: 4

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1
    maxSurge: 1
```

Meaning:

- At most 1 pod can be unavailable
- At most 1 extra pod can be created temporarily

4) Key Parameters

- maxUnavailable
Number of old pods that can go down during update.
- maxSurge
Extra pods allowed above desired replica count.
- Readiness Probe
Ensures traffic only goes to healthy pods.
- Liveness Probe
Restarts unhealthy pods.

5) Traffic Flow During Deployment

Load Balancer



Healthy Pods Only



Old + New versions coexist temporarily

Once new pod is Ready:

- It starts receiving traffic
- Old pod is removed

6) Advantages

- Zero downtime
- No traffic interruption
- Safe gradual rollout
- No duplicate environments required
- Resource-efficient

7) Limitations

- Old and new versions coexist → must be backward compatible
- DB schema changes must be handled carefully
- Slower rollout compared to recreate strategy

8) Data Consistency Considerations

When doing rolling update:

- ✓ Use backward-compatible APIs
- ✓ Avoid breaking DB changes
- ✓ Use feature flags if needed
- ✓ Apply DB migrations before deployment

Bad Example:

- New code expects new DB column
- Old pods still running → may break

9) Comparison With Other Strategies

Rolling Update:

- Gradual replacement
- No downtime
- Mixed versions temporarily

Blue-Green:

- Two separate environments
- Instant switch
- Higher infrastructure cost

Canary:

- Small % traffic to new version
- Risk-controlled rollout

10) Failure Handling

If new pod fails readiness:

- Kubernetes stops rollout
- Old pods continue serving traffic
- Rollback can be triggered

Deployment does NOT take down the entire system.

11) When To Use

- ✓ Stateless services
- ✓ High-availability APIs
- ✓ Microservices
- ✓ Kubernetes environments

Avoid if:

- ✗ Breaking schema changes
- ✗ Major incompatible changes

12) Interview-Ready Summary

Rolling Update is a zero-downtime deployment strategy where instances are gradually replaced with newer versions while keeping the service available. Traffic is routed only to healthy pods using readiness checks. It requires backward compatibility since old and new versions coexist temporarily. Kubernetes manages rollout using parameters like `maxUnavailable` and `maxSurge`, ensuring controlled and safe deployment.

=====

END

=====

Response code

=====

Important HTTP Response Codes & Meaning

=====

Status Code	Response Message	How to Set in ResponseEntity	When to Use
200	OK	<code>ResponseEntity.ok(body)</code>	Successful GET / PUT (with response body)
201	Created	<code>ResponseEntity.status(HttpStatus.CREATED).body(body)</code>	Resource created (POST)
202	Accepted	<code>ResponseEntity.status(HttpStatus.ACCEPTED).body(msg)</code>	Async processing (Kafka, @Async)
204	No Content	<code>ResponseEntity.noContent().build()</code>	Success but no response body (DELETE)
400	Bad Request	<code>ResponseEntity.badRequest().body(error)</code>	Invalid input / validation failure
401	Unauthorized	<code>ResponseEntity.status(HttpStatus.UNAUTHORIZED).body(msg)</code>	Missing/invalid authentication (JWT issue)
403	Forbidden	<code>ResponseEntity.status(HttpStatus.FORBIDDEN).body(msg)</code>	Authenticated but no permission (RBAC fail)
404	Not Found	<code>ResponseEntity.status(HttpStatus.NOT_FOUND).body(msg)</code>	Resource not found

405	Method Not Allowed	<code>ResponseEntity.status(HttpStatus.METHOD_NOT_ALLOWED).build()</code>	Wrong HTTP method
408	Request Timeout	<code>ResponseEntity.status(HttpStatus.REQUEST_TIMEOUT).body(msg)</code>	Client request timeout
409	Conflict	<code>ResponseEntity.status(HttpStatus.CONFLICT).body(msg)</code>	Duplicate data / business conflict
410	Gone	<code>ResponseEntity.status(HttpStatus.GONE).body(msg)</code>	Resource permanently deleted
415	Unsupported Media Type	<code>ResponseEntity.status(HttpStatus.UNSUPPORTED_MEDIA_TYPE).body(msg)</code>	Wrong content type (not JSON/XML)
422	Unprocessable Entity	<code>ResponseEntity.status(HttpStatus.UNPROCESSABLE_ENTITY).body(error)</code>	Validation errors (better than 400 in some APIs)
429	Too Many Requests	<code>ResponseEntity.status(HttpStatus.TOO_MANY_REQUESTS).body(msg)</code>	Rate limiting (API Gateway / IP limit)
500	Internal Server Error	<code>ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).body(msg)</code>	Generic server failure
501	Not Implemented	<code>ResponseEntity.status(HttpStatus.NOT_IMPLEMENTED).body(msg)</code>	Feature not supported
502	Bad Gateway	<code>ResponseEntity.status(HttpStatus.BAD_GATEWAY).body(msg)</code>	Downstream service failure
503	Service Unavailable	<code>ResponseEntity.status(HttpStatus.SERVICE_UNAVAILABLE).body(msg)</code>	Service down / maintenance
504	Gateway Timeout	<code>ResponseEntity.status(HttpStatus.GATEWAY_TIMEOUT).body(msg)</code>	Downstream timeout

Most Important Codes for Interviews

Must Remember:

200 → Success
201 → Created
204 → No Content
400 → Bad Request
401 → Unauthorized
403 → Forbidden
404 → Not Found
409 → Conflict
500 → Internal Server Error
503 → Service Unavailable

Difference Between Commonly Confused Codes

401 vs 403

401 → Not authenticated
403 → Authenticated but not authorized

400 vs 422

400 → Malformed request
422 → Validation/business rule failure

404 vs 403

404 → Resource doesn't exist
403 → Exists but no access

How Spring Sets Response Codes

Using `@ResponseStatus`:

`@ResponseStatus(HttpStatus.NOT_FOUND)`

Using ResponseEntity:

```
return ResponseEntity.status(404).body(...);
```

Using ErrorHandler:

```
@ExceptionHandler(...)
@ResponseStatus(HttpStatus.BAD_REQUEST)
```

=====

Interview-Ready Summary

=====

HTTP response codes are standardized 3-digit numbers that indicate the outcome of an HTTP request. They are grouped into 1xx informational, 2xx success, 3xx redirection, 4xx client errors, and 5xx server errors. Proper use of status codes improves API clarity, debugging, and contract correctness in distributed systems.

=====

END

=====

Service Boundry

Service Boundary – Brief Summary

A **service boundary** defines what responsibilities, data, and business capability belong inside a microservice.

Key Principles

- **Business Capability Based**
 - Each service represents one clear business function.
- **Owns Its Data**
 - A service has its own database.
 - No direct DB access from other services.
- **High Cohesion**
 - All functionalities inside the service are closely related.
- **Low Coupling**
 - Avoid excessive synchronous calls between services.
- **Independent Deployment**
 - Services should be deployable without coordinating with others.
- **Change Isolation**
 - Features that change together → same service.
 - Features that evolve independently → separate services.
- **Transaction Boundary**
 - Avoid frequent distributed transactions.
 - Prefer eventual consistency (Saga pattern).

One-Line Definition

> A service boundary defines the scope of responsibility and data ownership of a microservice to ensure high cohesion, low coupling, and independent scalability.

Custom Aspect

 `</> Java`

```
import java.lang.annotation.*;

@Target(ElementType.METHOD)
@Retention(RetentionPolicy.RUNTIME)
@Documented
public @interface CheckRole {
    String value(); // role required
}
```

Use Annotation on Method

 `</> Java`

```
import org.springframework.stereotype.Service;

@Service
public class OrderService {

    @CheckRole("ADMIN")
    public void createOrder() {
        System.out.println("Order Created");
    }
}
```

JDBC

2. JDBC Repository (Using Plain JDBC)

 Java 

```
import java.sql.*;
import java.util.ArrayList;
import java.util.List;

public class UserRepository {

    private static final String URL = "jdbc:mysql://localhost:3306/testdb";
    private static final String USER = "root";
    private static final String PASSWORD = "password";

    public List<UserDTO> getAllUsers() {

        List<UserDTO> users = new ArrayList<>();
        String query = "SELECT id, name, email, status FROM users";

        try (
            Connection connection = DriverManager.getConnection(URL, USER, PASSWORD);
            PreparedStatement ps = connection.prepareStatement(query);
            ResultSet rs = ps.executeQuery()
        ) {

            while (rs.next()) {
                UserDTO user = new UserDTO(
                    rs.getLong("id"),
                    rs.getString("name"),
                    rs.getString("email"),
                    rs.getString("status")
                );
                users.add(user);
            }

        } catch (SQLException e) {
            throw new RuntimeException("Error fetching users", e);
        }

        return users;
    }
}
```


Dynamic rules engine

Dynamic Rule Engine (Teammate+) – Complete Interview Explanation

1. Overview

In Teammate+, a **dynamic rule engine** is implemented to allow business logic to be configured without modifying application code. Instead of hardcoding conditions in Java services, rules are created through an **admin UI page**, stored in the **database**, and evaluated dynamically at runtime.

This design enables business teams or administrators to change system behavior without redeploying the application.

Typical use cases:

- Assessment validation rules
- Risk classification based on score
- Conditional workflow triggers
- Approval requirements
- Dynamic scoring logic
- Conditional notifications
- Role-based workflow actions

2. Problem With Hardcoded Business Logic

Without a rule engine, business logic is embedded directly in code.

Example:

```
```java
if(score < 40){
 assessment.setRiskLevel("HIGH");
}
```
```

Problems with this approach:

- Requires code changes for every rule update
- Requires deployment
- Business users cannot manage logic
- Difficult to scale rule complexity

A dynamic rule engine externalizes this logic into the database.

3. Core Idea of Dynamic Rule Engine

Rules are stored as ****data instead of code****.

Example rule stored in DB:

| rule_name | condition_expression | action_expression |
|----------------|----------------------|----------------------|
| HIGH_RISK_RULE | score < 40 | setRiskLevel("HIGH") |

At runtime:

1. Rules are loaded from the database
2. Conditions are evaluated dynamically
3. If the condition is true, the action is executed

4. High Level Architecture

Admin UI
↓
Rule Management API
↓
Database (Rules Table)
↓
Rule Engine Service
↓
Load Rules
↓
Evaluate Conditions
↓
Execute Actions
↓
Return Modified Entity

5. Rule Creation Flow

Step 1 — Admin Creates Rule From UI

| Field | Value |
|-------------|----------------------|
| Rule Name | High Risk Assessment |
| Entity Type | Assessment |
| Condition | score < 40 |
| Action | setRiskLevel("HIGH") |
| Priority | 10 |

| Active | true |

Step 2 — Rule Stored In Database

Example table:

```
```sql
CREATE TABLE rules (
 id BIGINT PRIMARY KEY,
 rule_name VARCHAR(255),
 entity_type VARCHAR(100),
 condition_expression TEXT,
 action_expression TEXT,
 priority INT,
 is_active BOOLEAN,
 created_at TIMESTAMP,
 updated_at TIMESTAMP
);
```
```

Example record:

| id | rule_name | entity_type | condition_expression | action_expression | priority | is_active |
|----|----------------|-------------|----------------------|----------------------|----------|-----------|
| 1 | HIGH_RISK_RULE | ASSESSMENT | score < 40 | setRiskLevel("HIGH") | 10 | true |

6. Rule Execution Flow

When an event occurs (example: assessment submission):

Assessment Submitted

↓

Rule Engine Service

↓

Fetch Active Rules From Database

↓

Filter Rules By Entity Type

↓

Sort Rules By Priority

↓

Evaluate Condition

↓

Execute Action

↓

Return Updated Object

7. Core Components

Rule Entity

```
```java
public class Rule {

 private Long id;
 private String ruleName;
 private String entityType;
 private String conditionExpression;
 private String actionExpression;
 private Integer priority;
 private Boolean active;
}
```
```

Rule Repository

```
```java
@Repository
public interface RuleRepository extends JpaRepository<Rule, Long> {

 List<Rule> findByEntityTypeAndActiveTrueOrderByPriorityDesc(String entityType);
}
```
```

Ensures:

- Only active rules are loaded
- Rules executed in priority order

Rule Engine Service

```
```java
@Service
public class RuleEngineService {

 @Autowired
 private RuleRepository ruleRepository;
}
```

```

public void evaluateRules(Assessment assessment){

 List<Rule> rules = ruleRepository
 .findByEntityTypeAndActiveTrueOrderByPriorityDesc("ASSESSMENT");

 for(Rule rule : rules){

 if(evaluateCondition(rule.getConditionExpression(), assessment)){
 executeAction(rule.getActionExpression(), assessment);
 }
 }
}
}
...

```

## ## 8. Condition Evaluation

Conditions stored as expressions.

Examples:

```

...
score < 40
department == 'Finance'
riskScore > 80
assessmentType == 'INTERNAL'
...

```

Evaluated using **\*\*Spring Expression Language (SpEL)\*\***.

```

```java
private boolean evaluateCondition(String expression, Assessment assessment){

    ExpressionParser parser = new SpelExpressionParser();

    EvaluationContext context = new StandardEvaluationContext(assessment);

    Boolean result = parser.parseExpression(expression).getValue(context, Boolean.class);

    return Boolean.TRUE.equals(result);
}
...

---

```

9. Action Execution

Actions executed when condition is satisfied.

Examples:

```
...
setRiskLevel("HIGH")
setApprovalRequired(true)
assignReviewer("RISK_TEAM")
sendNotification("ADMIN")
...
```

Example implementation:

```
```java
private void executeAction(String action, Assessment assessment){

 if(action.contains("setRiskLevel")){
 assessment.setRiskLevel("HIGH");
 }

 if(action.contains("setApprovalRequired")){
 assessment.setApprovalRequired(true);
 }
}
...
```
```

In advanced systems actions use **strategy pattern** or **handler classes**.

10. Rule Caching

Fetching rules from the database every request is inefficient.

Rules are cached in memory.

Application Startup

↓

Load Rules From Database

↓

Store In Memory Cache

↓

Runtime Rule Evaluation

Example cache:

```
...
Map<String, List<Rule>> ruleCache
...
```

Example:

```
...
ASSESSMENT -> [Rule1, Rule2, Rule3]
SURVEY -> [Rule4, Rule5]
...
```

Cache refresh strategies:

- Scheduled refresh
- Event-based refresh
- Manual refresh

11. Rule Priority Handling

Multiple rules may apply.

| Rule | Condition | Action |
|--------------------|------------|--------------------------|
| HIGH_RISK_RULE | score < 40 | setRiskLevel("HIGH") |
| CRITICAL_RISK_RULE | score < 30 | setRiskLevel("CRITICAL") |

Priority:

| Rule | Priority |
|--------------------|----------|
| CRITICAL_RISK_RULE | 20 |
| HIGH_RISK_RULE | 10 |

Higher priority executes first.

12. Rule Versioning

Track rule updates.

```
```sql
CREATE TABLE rule_versions (
 id BIGINT PRIMARY KEY,
 rule_id BIGINT,
 version INT,

```



```

 condition_expression TEXT,
 action_expression TEXT,
 created_at TIMESTAMP
);

```

Benefits:

- Audit history
- Rollback capability
- Track rule evolution

---

## ## 13. Rule Execution Logging

```

```sql
CREATE TABLE rule_execution_log (
    id BIGINT PRIMARY KEY,
    rule_id BIGINT,
    entity_id BIGINT,
    result VARCHAR(50),
    executed_at TIMESTAMP
);

```

Example log:

```

| rule_id | entity_id | result |
|-----|-----|-----|
| 1 | 4521 | SUCCESS |

```

14. Example End-to-End Scenario

Input:

```

...
score = 35
department = Finance
...

```

Rules:

```

| Rule | Condition | Action |
|----|-----|-----|
| Rule1 | score < 40 | setRiskLevel("HIGH") |

```

```
| Rule2 | department == 'Finance' | setApprovalRequired(true) |
```

Execution:

```
...
```

```
Rule1 triggered → riskLevel = HIGH
```

```
Rule2 triggered → approvalRequired = true
```

```
...
```

Final result:

```
...
```

```
riskLevel = HIGH
```

```
approvalRequired = true
```

```
...
```

```
---
```

15. Advantages

- Dynamic business logic
- No deployment required
- Business-friendly configuration
- Centralized rule management
- Extensible architecture

```
---
```

16. Limitations

- Complex debugging
- Rule conflicts
- Performance concerns for large rule sets

```
---
```

17. Interview Summary Answer

In Teammate+, a dynamic rule engine externalizes business logic from application code.

Administrators create rules via a UI where each rule contains a condition and action. These rules are stored in the database and evaluated dynamically at runtime using Spring Expression Language. The rule engine loads active rules, evaluates conditions, executes actions, and applies them to entities.

Rules are prioritized and cached in memory to improve performance. This allows business logic to change without modifying code or redeploying the application.

Hackthon

Hackathon Project: AI-Driven Risk Auditing(Code Games 2024 – 1st Place)

Summary:

Built an AI-powered enterprise risk intelligence platform to assist auditors in identifying similar historical risks and automatically generating mitigation controls. The system leveraged semantic vector search and large language models to improve risk discovery beyond traditional keyword-based search systems.

Core Technologies Used:

- Backend: Java, Spring Boot (REST APIs)
- AI/LLM: OpenAI GPT-3.5 Turbo
- Embeddings: OpenAI Text Embeddings
- Vector Database: Pinecone (vector embeddings)
- Frontend: React
- Architecture: RAG-style semantic retrieval + LLM generation

RAG -> retrieval augmented generation

Technical Implementation:

1. Semantic Risk Representation

Risk descriptions were converted into high-dimensional vector embeddings using OpenAI embedding models.

These embeddings capture semantic meaning, allowing detection of risks that are conceptually similar even when the wording differs.

2. Vector Storage and Retrieval

Embeddings were stored in the Pinecone vector database.

For each new risk submitted by an auditor:

- The risk text was converted to an embedding
- A similarity search (cosine similarity) was executed in Pinecone
- The system retrieved the top K most semantically similar historical risks

3. Semantic Risk Search Pipeline

New Risk Input



Embedding Generation (OpenAI API)



Vector Query in Pinecone



Top-K Similar Historical Risks



Context sent to LLM

4. AI Risk and Control Generation

The retrieved risks were used as context for GPT-3.5 Turbo to generate:

- New potential lookalike risks
- Recommended mitigation controls
- Compliance suggestions

Prompt structure included:

- current risk
- similar historical risks
- control examples

5. Backend API Flow

POST /risk/analyze

Steps:

1. Receive risk description
2. Generate embedding via OpenAI API
3. Query Pinecone for similar vectors
4. Send retrieved risks to GPT-3.5
5. Return generated risks and mitigation controls

6. Key Technical Concepts Implemented

Vector Embeddings

Transforms text into numeric vectors that represent semantic meaning.

Approximate Nearest Neighbor Search

Used by Pinecone to efficiently find similar vectors among large datasets.

Semantic Retrieval

Enables contextual similarity search rather than keyword matching.

LLM-Based Risk Generation

GPT-3.5 generated additional risks and suggested mitigation strategies using retrieved context.

7. System Outcome

The platform enabled:

- Semantic search across historical audit risks
- AI-generated risk discovery
- Automated control recommendations

This significantly improved the efficiency of risk analysis and audit preparation, which led to the project winning ****1st place in the ESG category at the Code Games 2024 hackathon****.

Kafka

topics

- 1) DLQueue
- 2) retry mechanism
- 3) fan out

=====

KAFKA SHORT NOTES

=====

1) TOPICS AND THEIR USE

- Topic = Logical category where messages are stored.
- Producer publishes to a topic.
- Consumers subscribe to a topic.
- Topic is divided into partitions.

Example:

order.created
order.payment.completed

Use:

- Decouple services
- Enable async communication
- Event-driven architecture

2) PUBLISHER SMALL SETUP (SPRING BOOT)

application.properties

spring.kafka.bootstrap-servers=localhost:9092

Producer Service

```
@Autowired
private KafkaTemplate<String, Object> kafkaTemplate;

public void publishOrder(Order order) {
    kafkaTemplate.send("order.created", order);
}
```

Flow:

Order Object

↓
JsonSerializer
↓
Kafka Broker
↓
Stored in Topic Partition

3) CONSUMER SMALL SETUP (SPRING BOOT)

application.properties

spring.kafka.bootstrap-servers=localhost:9092

Consumer Listener

```
@KafkaListener(
    topics = "order.created",
    groupId = "payment-group"
)
public void consumeOrder(Order order) {
    System.out.println(order);
}
```

Flow:

Consumer joins group

↓

Kafka assigns partition

↓

Polls messages

↓

Processes message

↓

Commits offset

4) PARTITIONING AND OFFSET

Partition:

- Topic is divided into partitions.
- Enables parallel processing.
- Ordering guaranteed only inside one partition.

Example:

Topic: order.created

Partition 0:

Offset 0 → Order1

Offset 1 → Order4

Partition 1:

Offset 0 → Order2

Offset 1 → Order5

Offset:

- Sequential number inside partition.
- Tracks consumer progress.
- Stored per (Consumer Group + Partition).

Rule:

Next read = Last committed offset + 1

5) SMALL END-TO-END FLOW

User places order



Order Service



Publish → order.created topic



Kafka stores message in Partition



Payment Service consumes



Payment successful



Publish → order.payment.completed



Delivery Service consumes

6) ZOOKEEPER AND KAFKA SERVER (LOCAL)

Old Setup:

ZooKeeper + Kafka Broker

ZooKeeper:

- Managed metadata
- Leader election
- Broker coordination

Kafka Broker:

- Stores messages
- Handles partitions
- Serves producers & consumers

Modern Setup (Recommended):

Kafka in KRaft mode (No ZooKeeper)

Local:

- Run Kafka via Docker
- Expose port 9092
- Connect via localhost:9092

=====
KEY FORMULA
=====

Partition → Parallelism

Consumer Group → Load balancing

Offset → Progress tracking

Kafka → Event backbone

=====

=====
1) RETRY MECHANISM IN KAFKA
=====

Retry is used when message processing fails temporarily
(DB issue, API failure, network error).

Instead of losing the message, the consumer retries processing.

Flow:

Producer
↓
Kafka Topic
↓
Consumer reads message
↓
Processing fails
↓
Retry message
↓
If retry limit reached → DLQ

Common Retry Approaches

1) Immediate Retry

- Retry inside consumer code.
- Not recommended for heavy workloads.

2) Retry Topics (Production Pattern)

```
order.created
  ↓ fail
order.created.retry.5s
  ↓ fail
order.created.retry.1m
  ↓ fail
order.created.dlq
```

3) Spring Kafka Retry

```
@RetryableTopic(
    attempts = 3,
    backoff = @Backoff(delay = 5000)
)
```

Used when failures are temporary.

=====

2) OFFSET COMMITTING

=====

Offset = position of a message inside a partition.

Kafka uses offset to track which messages a consumer has processed.

Structure:

Consumer Group + Partition + Offset

Example:

Partition 0

Offset 0 → Order1

Offset 1 → Order2

Offset 2 → Order3

If offset 1 committed → next read starts from offset 2.

Types of Offset Commit

1) Auto Commit

`enable.auto.commit=true`

Kafka automatically commits offsets periodically.

Risk:

Message may be lost if processing fails after commit.

2) Manual Commit (Recommended)

Process message

↓

Commit offset

Spring Example:

`ack.acknowledge()`

Ensures message processed before committing offset.

=====

3) DLQ (DEAD LETTER QUEUE)

=====

DLQ stores messages that cannot be processed even after multiple retries.

Purpose:

- Prevent message loss
- Allow debugging
- Prevent blocking consumer

Flow:

`order.created`

↓ fail

Retry Topic

↓ fail

Retry Topic

↓ fail

`order.created.dlq`

DLQ message usually contains:

- original event
- error reason
- retry count
- timestamp

Engineers later inspect DLQ and decide to:

- reprocess
- fix data
- discard message

=====

4) EVENT ID IN KAFKA

=====

Event ID is a unique identifier for each message.

Kafka itself does NOT generate event IDs.
The producer service generates them.

Usually generated using:

```
UUID.randomUUID()
```

Example event:

```
{  
  "eventId": "a1b2c3",  
  "orderId": "ORD123",  
  "status": "CREATED"  
}
```

Purpose:

- prevent duplicate processing
- track events
- enable idempotency

=====

5) DUPLICATE EVENT ID HANDLING

=====

Kafka can sometimes deliver duplicate messages due to:

- consumer retry

- network failure
- rebalance
- producer retry

To avoid processing duplicates, consumers implement IDEMPOTENCY.

Approach:

Before processing event:

Check if eventId is already processed.

Flow:

```

Consumer receives event
    ↓
Check eventId in storage
    ↓
If exists → ignore
If not exists → process event
    ↓
Store eventId
  
```

Where eventId is stored:

1) Database table

```

processed_events
-----
event_id
processed_at
  
```

2) Redis (for high throughput)

```

key = processed_event:{eventId}
TTL = 24 hours
  
```

Example logic:

```

if(eventId exists)
    skip processing
else
    process event
    store eventId
  
```

=====

SUMMARY

=====

Retry → handle temporary failures
Offset → track consumer progress
DLQ → store permanently failed messages
Event ID → uniquely identify event
Duplicate handling → ensure idempotent processing

=====

KAFKA DLQ (Dead Letter Queue) - SHORT NOTE

=====

1) WHEN DOES A MESSAGE GO TO DLQ?

A message is sent to DLQ when the consumer fails to process it after retries.

Common reasons:

- Validation failure (invalid payload)
- Deserialization error (JSON parsing issue)
- Business rule failure (missing required fields)
- External dependency failure (DB/API down)
- Retry attempts exhausted

2) HOW DOES IT GO TO DLQ? (SPRING KAFKA)

Spring Kafka uses:

- DefaultErrorHandler
- DeadLetterPublishingRecoverer

Flow:

Consumer → Retry (e.g., 3 times) → Still fails → Sent to DLQ topic

Example config:

@Bean

```
public DefaultErrorHandler errorHandler(KafkaTemplate<String, Object> template) {  
    DeadLetterPublishingRecoverer recoverer =  
        new DeadLetterPublishingRecoverer(template);  
  
    DefaultErrorHandler handler =  
        new DefaultErrorHandler(recoverer, new FixedBackOff(1000L, 3));  
    return handler;  
}
```

```
}
```

3) EXAMPLE SCENARIO

Message:

```
{  
  "auditId": null  
}
```

Consumer logic:

if (auditId == null) throw Exception

Flow:

main-topic → consumer ❌ → retry → fail → DLQ topic

4) HOW TO FETCH MESSAGE FROM DLQ?

Option 1: Kafka Consumer

```
@KafkaListener(topics = "main-topic.DLT")  
public void readDlq(String message) {  
    System.out.println("DLQ message: " + message);  
}
```

Option 2:

- Kafka UI tools
- CLI consumer

5) HOW TO REPLAY A MESSAGE FROM DLQ?

Replay = produce message again to main topic

```
@KafkaListener(topics = "main-topic.DLT")  
public void replay(String message,  
    @Header(KafkaHeaders.RECEIVED_PARTITION_ID) int partition,  
    @Header(KafkaHeaders.RECEIVED_KEY) String key) {  
  
    kafkaTemplate.send("main-topic", key, message);  
}
```

6) PARTITION HANDLING DURING REPLAY

Case 1: No partition specified

→ Kafka assigns:

- Based on key (hashing)
- Or round-robin

Case 2: Preserve original partition (BEST PRACTICE)

```
kafkaTemplate.send(  
    new ProducerRecord<>("main-topic", partition, key, message)  
);
```

→ Message goes to SAME partition

7) OFFSET HANDLING

- Original offset is NOT reused
- Replay creates NEW message

Example:

Before:

Partition 0 → Offset 10 (failed)

After replay:

Partition 0 → Offset 25 (new message)

8) HOW TO HANDLE FAILED MESSAGE PROPERLY

Best practices:

1. Idempotency

- Use unique eventId
- Avoid duplicate processing

2. Retry limit

- Prevent infinite loops

3. Logging

- Store error details

4. DLQ metadata

- original topic, partition, offset

5. Alerting

- Monitor DLQ size

9) FINAL FLOW SUMMARY

Producer → Main Topic → Consumer

→ (Fail) → Retry → Fail

→ DLQ Topic

→ Ops Fix / Auto Replay

→ Produce Again → Main Topic

→ Consumer (Success)

KEY POINT:

Replay ≠ same message retry

Replay = NEW message with same data

##End to End flow

Audit Service

↓ (produces)

audit-events

↓

Rule Engine (consumer)

↓ (produces)

rule-events

↓

Notification Service (consumer)

How do you handle messages in Dead Letter Queue (DLQ) in your system?

We don't directly modify messages in Kafka. When a message goes to DLQ, we store it in a database and expose it via an Admin UI used by Ops.

Ops can review the failed payload, make necessary corrections, and trigger a retry.

The UI calls a backend API, which validates the data and republishes the message to the original Kafka topic.

We also maintain retry count and status to avoid infinite retries.

In non-production, we usually handle DLQ messages using Postman. Once we identify the issue and fix the data or code, we hit a retry API via Postman, which republishes the message to the original Kafka topic. Since it's non-prod, we don't rely on a full Admin UI and keep the process simple for faster debugging.

In our audit system, we use Kafka to publish audit events like finding creation. Multiple services like notification, dashboard, and rule engine consume these events asynchronously. We implemented retry with backoff and DLQ handling to ensure fault tolerance and allow manual replay of failed events.

◆ Who consumes the topic & why

- **Notification Service**

- Consumes: `audit.finding.created`
 - Purpose: Send email / in-app alerts to auditors, managers
 - Why async: Notification failure should not block audit creation
-

- **Dashboard Service**

- Consumes: `audit.finding.created`
 - Purpose: Update real-time metrics (open findings, severity count)
 - Why async: UI/dashboard updates can happen eventually (not critical path)
-

- **Rule Engine Service**

- Consumes: `audit.finding.created`
- Purpose: Evaluate dynamic rules (e.g., HIGH severity → escalate)
- Why async: Rule processing can be heavy and should be decoupled

-
- **Audit History / Logging Service (optional but realistic)**
 - Consumes: `audit.finding.created`
 - Purpose: Maintain audit trail / event history
 - Why async: Logging should not impact main transaction

◆ **Kafka Message Failure Summary**

- **Email sending failed**
 - SMTP server down / timeout / invalid email config
- **Database operation failed**
 - DB connection issue / deadlock / query timeout
- **Invalid event data**
 - Missing or null fields (e.g., `severity = null`) causing runtime exceptions
- **Code-level exception**
 - Bug in consumer logic (null pointer, divide by zero, etc.)
- **Downstream service failure**
 - Dependent microservice API is down or not responding
- **Schema mismatch / event format change**
 - Producer and consumer not aligned on event structure
- **Processing timeout / long execution**
 - Heavy logic causing consumer timeout or rebalance interruption

◆ **Key Understanding (important)**

- **Retry works for:** temporary issues (DB down, API failure)
- **DLQ needed for:** permanent issues (bad data, code bug, schema mismatch)

CI/CD

Docker

Kubernetes

Local Development

MICROSERVICE LOCAL RUN SETUP (SHORT NOTES)

PROJECT STRUCTURE

```
-----  
repo/  
├── services/  
│   ├── user-service/  
│   ├── order-service/  
│   └── payment-service/  
├── k8s/  
│   ├── user-service/  
│   ├── order-service/  
│   └── payment-service/  
└── start-local.ps1
```

WHAT HAPPENS AFTER GIT PULL

- ```

```
1. Start local Kubernetes cluster  
minikube start
  2. Use Minikube docker environment  
minikube docker-env
  3. Build docker images for services
  4. Deploy Kubernetes manifests  
kubectl apply -f k8s/
  5. Services communicate using Kubernetes DNS  
Example:  
http://user-service:8080

### OPTION 1 — RUN ALL SERVICES

```

```

Command:

```
.\start-local.ps1
or
.\start-local.ps1 all
```

What happens:

- Script scans "services/" directory
- Builds docker image for each service

- Deploys all k8s manifests
- Kubernetes creates pods and services

Example running pods:

user-service-pod  
order-service-pod  
payment-service-pod

Inter-service communication:

order-service → http://user-service:8080

## OPTION 2 — RUN SINGLE MICROSERVICE

-----  
Command:

.\start-local.ps1 order-service

What happens:

- Only order-service image is built
- Only order-service k8s manifest applied
- Other services are assumed to exist in cluster  
OR run in remote dev environment

Example flow:

order-service (local pod)  
↓  
http://user-service:8080  
↓  
user-service pod

## SERVICE COMMUNICATION

-----  
Services never call IPs.

They use the Kubernetes service name.

Example:

http://user-service:8080  
http://payment-service:8080

Kubernetes DNS automatically resolves.

## PORT MANAGEMENT

-----

Inside container:

`server.port=8080`

Kubernetes service mapping:

`port: 8080`

`targetPort: 8080`

Flow:

order-service pod



user-service service



user-service pod

## DEVELOPER WORKFLOW

-----

Fresh pull:

`git pull`

`.\start-local.ps1 order-service`

or

`git pull`

`.\start-local.ps1 all`

## KEY IDEA

-----

Script supports two modes:

- 1) Run full microservice system
- 2) Run only the service you are developing

DTO DAO



## STUDENT APP – SHORT NOTES (SPRING BOOT STRUCTURE)

### 1. PROJECT STRUCTURE

Typical layered architecture used in Spring Boot applications.

controller → Handles HTTP requests

service → Business logic

repository → Database interaction

entity → JPA entities mapped to DB tables

dto → Objects used for API request/response

mapper → Converts Entity ↔ DTO

Example package structure:

```
com.studentapp
├── controller
│ └── StudentController
├── service
│ ├── StudentService (interface)
│ └── StudentServiceImpl
├── repository
│ └── StudentRepository
├── entity
│ └── Student
├── dto
│ ├── StudentPersonalDTO
│ └── StudentCourseDTO
├── mapper
└── StudentMapper
```

---

### 2. ENTITY CLASS

Represents the database table.

```
@Entity
@Table(name="students")
class Student {
```

```
@Id
@GeneratedValue
private Long id;
```

```
private String name;
private String email;
private String schoolName;
```

```
private List<String> courses;
```

```
}
```

---

### 3. DTO CLASSES

DTO is used to send only required data to clients instead of exposing entities.

StudentPersonalDTO

```
class StudentPersonalDTO {
```

```
 private Long id;
 private String name;
 private String email;
```

```
}
```

StudentCourseDTO

```
class StudentCourseDTO {
```

```
 private String schoolName;
 private List<String> courses;
```

```
}
```

---

### 4. REPOSITORY (DAO)

Handles database operations.

@Repository

```
interface StudentRepository extends JpaRepository<Student, Long> {
```

```
}
```

---

### 5. SERVICE INTERFACE

Defines business operations.

```
interface StudentService {
```

```
StudentPersonalDTO getStudentPersonal(Long id);

StudentCourseDTO getStudentCourses(Long id);

}
```

---

## 6. SERVICE IMPLEMENTATION

Mapping using Java Streams.

```
@Service
class StudentServiceImpl implements StudentService {

 private StudentRepository repo;

 public StudentPersonalDTO getStudentPersonal(Long id) {

 Student student = repo.findById(id).orElseThrow();

 return List.of(student)
 .stream()
 .map(s -> new StudentPersonalDTO(
 s.getId(),
 s.getName(),
 s.getEmail()
))
 .findFirst()
 .orElse(null);
 }

 public StudentCourseDTO getStudentCourses(Long id) {

 Student student = repo.findById(id).orElseThrow();

 return List.of(student)
 .stream()
 .map(s -> new StudentCourseDTO(
 s.getSchoolName(),
 s.getCourses()
))
 .findFirst()
 .orElse(null);
 }

}
```

---

## 7. CONTROLLER

Exposes REST APIs.

```
@RestController
@RequestMapping("/students")
class StudentController {

 private StudentService service;

 @GetMapping("/personal/{id}")
 public StudentPersonalDTO getPersonal(@PathVariable Long id) {
 return service.getStudentPersonal(id);
 }

 @GetMapping("/courses/{id}")
 public StudentCourseDTO getCourses(@PathVariable Long id) {
 return service.getStudentCourses(id);
 }

}
```

---

## 8. WHY WE USE STREAM MAPPING

Instead of manual loops, Java Streams provide a functional and cleaner way.

Traditional approach

```
for(Student s : students){
 dtoList.add(convertToDTO(s));
}
```

Stream approach

```
students.stream()
 .map(StudentMapper::toDTO)
 .collect(Collectors.toList());
```

Advantages

- Cleaner and more readable code
- Functional programming style
- Easy transformation (Entity → DTO)
- Supports parallel processing

- Reduces boilerplate loops
  - Commonly used in microservice applications
- 

## 9. REAL PROJECT FLOW

Client → Controller → Service → Repository → Database

Response Flow

Database → Entity → Service → DTO Mapping → Controller → Client

---

## 10. WHY DTO IS USED

- Avoid exposing entity directly
- Control API response structure
- Improve security
- Reduce unnecessary data transfer
- Maintain separation between DB and API layer

# Circuit Breaker

```
=====
Circuit breaker config
=====
```

```
=====
RESILIENCE4J CIRCUIT BREAKER - SETUP NOTES
=====
```

## PURPOSE

-----

Circuit Breaker protects an application from repeatedly calling a failing service. If a service fails frequently, the circuit opens and prevents further calls, redirecting requests to a fallback method.

Example:

Primary Service : Redis

Fallback Service: Database

```

1. ADD DEPENDENCY (MAVEN)

```

```
<dependency>
 <groupId>org.springframework.cloud</groupId>
 <artifactId>spring-cloud-starter-circuitbreaker-resilience4j</artifactId>
</dependency>
```

```

2. ENABLE ACTUATOR (FOR MONITORING)

```

```
<dependency>
 <groupId>org.springframework.boot</groupId>
 <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
```

```

3. CONTROLLER IMPLEMENTATION

```

```
package in.ashokit.rest;
```

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;
```

```
import io.github.resilience4j.circuitbreaker.annotation.CircuitBreaker;
```

```

@RestController
public class DemoRestController {

 @GetMapping("/data")
 @CircuitBreaker(
 name = "ashokit",
 fallbackMethod = "getDataFromDB"
)
 public String getDataFromRedis() {

 System.out.println("Calling Redis Service...");

 // Simulating failure
 int i = 10 / 0;

 return "Data fetched from Redis";
 }

 // FALLBACK METHOD
 // Must have same return type
 // Must accept Throwable parameter

 public String getDataFromDB(Throwable t) {

 System.out.println("Fallback triggered: calling DB");

 return "Data fetched from Database";
 }
}

```

---

#### 4. APPLICATION.YML CONFIGURATION

---

```

server:
 port: 8081

spring:
 application:
 name: resilience4j-demo

management:
 endpoints:
 web:

```



exposure:  
include: '\*'

endpoint:  
health:  
show-details: always

health:  
circuitbreakers:  
enabled: true

resilience4j:  
circuitbreaker:  
configs:  
default:  
  
registerHealthIndicator: true  
  
slidingWindowSize: 10  
minimumNumberOfCalls: 5  
  
permittedNumberOfCallsInHalfOpenState: 3  
  
automaticTransitionFromOpenToHalfOpenEnabled: true  
  
waitDurationInOpenState: 30s  
  
failureRateThreshold: 50  
  
eventConsumerBufferSize: 10

---

## 5. IMPORTANT CONFIGURATION PARAMETERS

---

slidingWindowSize

Number of requests used to calculate failure rate.

Example:  
slidingWindowSize = 10  
Circuit breaker tracks last 10 calls.

minimumNumberOfCalls

Minimum calls required before calculating failure rate.

Example:

minimumNumberOfCalls = 5

failureRateThreshold

-----

Percentage of failures allowed before opening circuit.

Example:

failureRateThreshold = 50

If more than 50% requests fail → circuit opens.

waitDurationInOpenState

-----

Time circuit stays OPEN before testing service again.

Example:

waitDurationInOpenState = 30s

permittedNumberOfCallsInHalfOpenState

-----

Number of test calls allowed when the circuit is HALF\_OPEN.

Example:

3 test requests allowed.

## 6. CIRCUIT BREAKER STATES

-----

### 1. CLOSED

-----

Normal operation.

Requests go to primary service.

### 2. OPEN

-----

Service failure detected.

Primary service calls are blocked.

The fallback method is executed.

### 3. HALF\_OPEN

-----  
After wait duration expires.  
Limited calls are allowed to test service.

If successful → circuit closes.  
If failure → circuit opens again.

### 7. REQUEST FLOW

CLIENT  
|  
v  
CIRCUIT BREAKER  
|  
v  
PRIMARY SERVICE (REDIS)  
|  
| success  
v  
RESPONSE TO CLIENT

If failure occurs:

CLIENT  
|  
v  
CIRCUIT BREAKER  
|  
v  
PRIMARY SERVICE FAILS  
|  
v  
FALLBACK METHOD  
|  
v  
DATABASE RESPONSE

### 8. MONITORING CIRCUIT BREAKER

-----  
Check circuit breaker status using actuator:

http://localhost:8081/actuator/health

or

http://localhost:8081/actuator/circuitbreakers

---

## 9. KEY INTERVIEW POINT

---

Circuit Breaker prevents cascading failures in microservices by temporarily stopping calls to a failing service and redirecting requests to a fallback mechanism.

---

States of circuit breaker

=====

WHICH LOGIC RUNS IN WHICH STATE

---

STATE	PRIMARY LOGIC (REDIS)	FALLBACK
-------	-----------------------	----------

CLOSED	YES	Only if exception occurs
--------	-----	--------------------------

OPEN	NO	Always executed
------	----	-----------------

HALF_OPEN	Limited requests	If those requests fail
-----------	------------------	------------------------

=====

=====

=====

=====

# SCENARIO 3: Notification Service Failure

## 1. Context

---

- After key actions in the audit system:
  - Audit assigned
  - Finding approved
  - Status changed
- Backend triggers notification service:
  - Email service

## 2. Problem

---

- Notification service is down / slow
- API call to notification fails
- Without protection:
  - Main business flow also fails
  - User action (approve/assign) gets blocked

## 3. Circuit Breaker Usage

---

- Wrap notification call with circuit breaker

Example:

```
@CircuitBreaker(name = "notificationService", fallbackMethod = "notificationFallback")
```

## 4. Behavior

---

- After multiple failures → circuit opens
- System stops calling notification service
- Immediately executes fallback

## 5. Fallback Strategy

---

- Store notification event in:
  - Retry table (DB)
  - OR internal queue
- Scheduler / batch job retries later

## 6. Outcome

---

- Business operation SUCCESS (audit continues)
  - Notification becomes eventually consistent
  - System avoids unnecessary failures
7. Key Point

- 
- Non-critical services (like notifications) should NEVER break main flow



## SCENARIO 4: Rule Engine External Dependency Failure

### 1. Context

- 
- Rule engine evaluates:
    - Dynamic audit rules
    - Conditions based on JSON config
  - May depend on:
    - External config service
    - Remote API for enrichment

### 2. Problem

- 
- External dependency is slow or unavailable
  - Rule evaluation gets delayed or fails
  - Can block:
    - Audit creation
    - Finding processing

### 3. Circuit Breaker Usage

- 
- Wrap external calls inside rule engine

Example:

```
@CircuitBreaker(name = "ruleEngineService", fallbackMethod = "ruleFallback")
```

### 4. Behavior

---

- On repeated failures → circuit opens
  - External calls are skipped
  - Fallback logic is triggered immediately
5. Fallback Strategy
- 

- Use:
    - Default rules
    - Cached rule configurations
  - OR skip non-critical validations
6. Outcome
- 

- Audit workflow continues without interruption
  - Some rules may not apply (degraded mode)
  - System remains available and stable
7. Key Point
- 

- Critical flows should not depend fully on external rule evaluation
- Always design rule engine with fallback capability

# Optimistic vs pesimistic lock



## OPTIMISTIC LOCKING (JPA / HIBERNATE)

### Definition

Optimistic locking is a concurrency control mechanism used to prevent lost updates when multiple transactions try to update the same database row simultaneously. It assumes conflicts are rare and detects them during update instead of locking the row.

### How It Works

Optimistic locking uses a VERSION column in the table. Every time the row is updated, the version value is incremented. During update, Hibernate checks if the version in DB matches the version read earlier.

If version mismatch occurs → OptimisticLockException is thrown.

## ENTITY EXAMPLE

@Entity

```
public class Student {
```

```
 @Id
```

```
 private Long id;
```

```
 private String name;
```

```
 private int marks;
```

```
 @Version
```

```
 private Integer version;
```

```
}
```

## DATABASE TABLE

student

```

id | name | marks | version

1 | Tejas | 80 | 1
```

## INTERNAL SQL GENERATED BY HIBERNATE

UPDATE student

SET marks = ?, version = version + 1

WHERE id = ? AND version = ?

## CONCURRENT UPDATE SCENARIO

### Step 1

Thread A reads version = 1

Thread B reads version = 1

### Step 2

Thread A updates record

version becomes 2

### Step 3

Thread B tries update with version = 1

UPDATE student

SET marks = 90

WHERE id = 1 AND version = 1

No rows updated → Hibernate throws OptimisticLockException

## JDBC LEVEL INTERNAL FLOW

executeUpdate()

↓

if rowsUpdated == 0

↓

OptimisticLockException

## KEY CHARACTERISTICS

- Uses VERSION column in database
- No row-level database locking
- Multiple reads allowed
- Conflict detected during update
- Prevents lost update problem
- Improves performance in high-read systems

## WHEN TO USE

- Read-heavy applications
- Systems with low conflict probability
- Microservices and enterprise applications
- Audit systems, order systems, inventory updates

## INTERVIEW ONE-LINE ANSWER

Optimistic locking prevents concurrent update conflicts using a version column. During update, Hibernate checks the version value in the WHERE clause, and if the version has changed since it was read, an OptimisticLockException is thrown.

## PESSIMISTIC LOCKING (JPA / DATABASE)

### Definition

Pessimistic locking is a concurrency control mechanism where the database row is locked when it is read so that other transactions cannot modify it until the current transaction completes.

It assumes conflicts are likely, so the system locks the data immediately.

---

### WHERE WE APPLY IT

In Spring Boot / JPA it is applied in the repository layer using @Lock annotation and works inside a transaction.

### Example Repository

#### @Repository

```
public interface StudentRepository extends JpaRepository<Student, Long> {
```

```
 @Lock(LockModeType.PESSIMISTIC_WRITE)
 @Query("select s from Student s where s.id = :id")
 Student findStudentForUpdate(Long id);
}
```

---

### SERVICE LAYER

#### @Transactional

```
public void updateMarks(Long id){

 Student s = repo.findStudentForUpdate(id);

 s.setMarks(90);
}
```

### Transaction Flow

#### Transaction start

↓  
SELECT query with lock  
↓  
Row gets locked  
↓  
Update happens  
↓  
Transaction commit  
↓  
Lock released

---

## SQL GENERATED BY DATABASE

```
SELECT * FROM student WHERE id = ? FOR UPDATE
```

This locks the selected row until the transaction finishes.

---

## ROW LEVEL LOCKING

Pessimistic locking locks only the selected row, not the entire table.

### Example

Thread A locks row id=1  
Thread B can still update row id=2

---

## TYPES OF PESSIMISTIC LOCK

### 1. PESSIMISTIC\_READ

```
@Lock(LockModeType.PESSIMISTIC_READ)
```

#### Behavior

- Other transactions can read the row
- Updates are blocked until transaction completes

### 2. PESSIMISTIC\_WRITE

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
```

#### Behavior

- Prevents other transactions from updating the row
- Other transactions trying to lock or update must wait

### 3. PESSIMISTIC\_FORCE\_INCREMENT

- Works with versioned entities
- Locks the row and increments version

---

#### WHEN TO USE

- Banking transactions
- Payment processing
- Inventory management
- Ticket booking systems
- Critical data updates

---

#### KEY POINTS

- Implemented using database row locks
- Uses SQL "SELECT ... FOR UPDATE"
- Works inside transactions
- Prevents concurrent updates
- Can lead to deadlocks if not handled properly

# Exceptions

Exception	Type	Cause (When it happens)
SQLException	Checked (Exception → SQLException)	Database connection failure, wrong SQL query, DB down
IOException	Checked (Exception → IOException)	File read/write failure, network stream issue
FileNotFoundException	Checked (IOException → FileNotFoundException)	File path incorrect or file missing
ParseException	Checked (Exception → ParseException)	Date or string parsing fails
InterruptedException	Checked (Exception → InterruptedException)	Thread interrupted during sleep/wait
ClassNotFoundException	Checked (Exception → ClassNotFoundException)	Class loader cannot find a class
MalformedURLException	Checked (IOException → MalformedURLException)	Invalid URL format
InstantiationException	Checked (Exception → InstantiationException)	Object creation using reflection fails
NoSuchMethodException	Checked (Exception → NoSuchMethodException)	Method not found during reflection
InvocationTargetException	Checked (Exception → InvocationTargetException)	Exception thrown during reflection method invocation

Exception	Type	Cause (When it happens)
NullPointerException	Unchecked (RuntimeException → NullPointerException)	Calling method on null object
IllegalArgumentException	Unchecked (RuntimeException → IllegalArgumentException)	Passing invalid parameter to method
NumberFormatException	Unchecked (IllegalArgumentException → NumberFormatException)	Converting invalid string to number
IndexOutOfBoundsException	Unchecked (RuntimeException → IndexOutOfBoundsException)	Accessing invalid index in array/list
ArithmeticException	Unchecked (RuntimeException → ArithmeticException)	Division by zero
ClassCastException	Unchecked (RuntimeException → ClassCastException)	Casting object to incompatible type
IllegalStateException	Unchecked (RuntimeException → IllegalStateException)	Method called at wrong time/state
UnsupportedOperationException	Unchecked (RuntimeException → UnsupportedOperationException)	Operation not supported
ConcurrentModificationException	Unchecked (RuntimeException → ConcurrentModificationException)	Modifying collection during iteration
ArrayStoreException	Unchecked (RuntimeException → ArrayStoreException)	Wrong type stored in array



transactional DeadLock

## Deadlock in Transaction

Deadlock occurs when two or more transactions hold locks on resources and each transaction waits for the other to release the lock. Because of this, none of the transactions can proceed and the database detects a deadlock and aborts one transaction.

### Common Causes of Deadlock

#### 1. Updating rows in different order

Example:

Transaction A updates row 1 then row 2

Transaction B updates row 2 then row 1

Both transactions wait for each other's lock.

#### 2. Long running transactions

If a transaction holds a lock for a long time (external API call, heavy processing), other transactions may wait and cause deadlock.

#### 3. Pessimistic locking

Using `SELECT ... FOR UPDATE` or JPA `PESSIMISTIC_WRITE` can create lock conflicts between concurrent transactions.

#### 4. Missing database indexes

Without indexes, database may scan and lock many rows or even table level locks which increases deadlock chances.

#### 5. Bulk updates or deletes

Large updates inside a transaction can lock many rows.

#### 6. Foreign key cascading operations

Updating parent and child tables in different order by different transactions can lead to deadlock.

### How to Avoid Deadlock

#### 1. Access tables and rows in the same order in all transactions.

#### 2. Keep transactions short.

Do not call external APIs or long processing inside transactions.

#### 3. Use proper indexes to avoid locking large number of rows.

#### 4. Prefer optimistic locking when possible.

Example:

`@Version`

`private Long version;`

5. Avoid unnecessary pessimistic locks.

6. Implement retry mechanism if deadlock occurs.

Common Exception in Spring

`org.springframework.dao.DeadlockLoserDataAccessException`

Sonarqube

=====

## SONARQUBE – COMPLETE NOTES (REAL PROJECT)

=====

### 1) WHAT IS SONARQUBE

-----

SonarQube is a Static Code Analysis tool used to continuously inspect code quality, security vulnerabilities, bugs, and maintainability issues.

It scans the source code without executing the application.

Goal:

Improve code quality and prevent bad code from reaching production.

Main capabilities:

- Detect Bugs
- Detect Security Vulnerabilities
- Detect Code Smells
- Measure Code Coverage
- Detect Code Duplication
- Calculate Technical Debt
- Enforce Quality Gates
- Integrate with CI/CD pipelines

Example technologies supported:

Java  
Spring Boot  
Python  
JavaScript  
TypeScript  
C#  
C++  
Go

-----

### 2) SONARQUBE ARCHITECTURE (REAL APPLICATION)

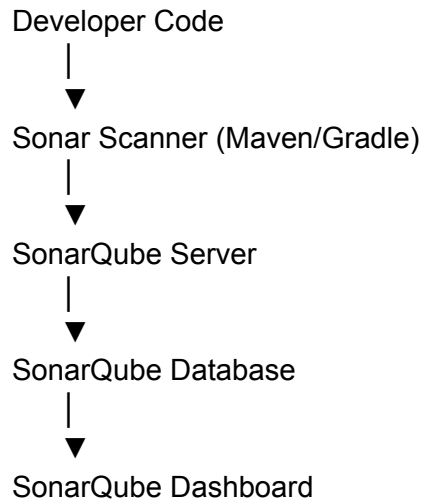
-----

Main components:

1) SonarQube Server

- 2) Sonar Scanner
- 3) SonarQube Database
- 4) Web Dashboard

#### Architecture Flow



#### Components explanation

##### Sonar Scanner

-----  
Tool that scans the source code.

Examples:

sonar-scanner  
mvn sonar:sonar  
gradle sonarqube

##### SonarQube Server

-----  
Processes analysis results and applies rules.

##### SonarQube Database

-----  
Stores:

- issues
- metrics
- coverage
- technical debt
- history

Dashboard

-----

Web UI used by developers to see issues.

-----

### 3) HOW SONARQUBE IS CONFIGURED IN REAL PROJECT

-----

#### Step 1 – Install SonarQube Server

Download SonarQube from

<https://www.sonarqube.org/>

Run server:

bin/windows-x86-64/StartSonar.bat

Server runs on:

<http://localhost:9000>

Default login

username: admin

password: admin

-----

#### Step 2 – Install Sonar Scanner

Download:

<https://docs.sonarsource.com/sonarqube/latest/analyzing-source-code/scanners/sonarscanner/>

Add to PATH

---

### Step 3 – Configure project (Maven Example)

pom.xml

```
<plugin>
 <groupId>org.sonarsource.scanner.maven</groupId>
 <artifactId>sonar-maven-plugin</artifactId>
 <version>3.9.1</version>
</plugin>
```

---

### Step 4 – Create sonar-project.properties

```
sonar.projectKey=my-project
sonar.projectName=MyProject
sonar.projectVersion=1.0
```

```
sonar.sources=src/main/java
sonar.tests=src/test/java
```

```
sonar.java.binaries=target/classes
```

```
sonar.host.url=http://localhost:9000
sonar.login=SONAR_TOKEN
```

---

### Step 5 – Run analysis

```
mvn clean verify sonar:sonar
```

OR

```
sonar-scanner
```



After execution results appear in SonarQube dashboard.

---

#### 4) TYPES OF ISSUES DETECTED BY SONARQUBE

---

SonarQube categorizes issues into 4 types.

##### 1) BUG

Definition

Code that may cause runtime failure.

Example

```
String name = null;
System.out.println(name.length());
```

Issue detected

Possible NullPointerException

---

##### 2) VULNERABILITY (Security Issues)

Definition

Security weakness in code.

Example

```
String query =
"SELECT * FROM users WHERE id=" + userId;
```

Issue

SQL Injection

Fix

Use PreparedStatement

---

### 3) CODE SMELL

Definition

Bad coding practice that affects maintainability.

Example

```
if(isValid == true)
```

Better

```
if(isValid)
```

Another example

```
public void method() {

 if(a==1) {
 if(b==2) {
 if(c==3) {
 if(d==4) {
 }
 }
 }
 }
}
```

Issue

High Cyclomatic Complexity

-----

#### 4) SECURITY HOTSPOTS

Definition

Code that may be risky and requires manual review.

Example

```
File file = new File(userInputPath);
```

Risk

Path Traversal

-----

#### 5) METRICS SHOWN IN SONARQUBE DASHBOARD

-----

Main metrics:

Bugs

----

Real defects.

Vulnerabilities

-----

Security problems.

Code Smells

-----

Maintainability issues.

## Coverage

-----

Unit test coverage.

## Duplication

-----

Repeated code blocks.

## Technical Debt

-----

Estimated time required to fix issues.

Example dashboard

Bugs: 2

Vulnerabilities: 1

Code Smells: 15

Coverage: 68%

Duplications: 4%

-----

## 6) CODE COVERAGE IN SONARQUBE

-----

SonarQube itself does NOT generate coverage.

It reads coverage reports from tools like:

Jacoco

Cobertura

Example Jacoco integration

```
<plugin>
 <groupId>org.jacoco</groupId>
 <artifactId>jacoco-maven-plugin</artifactId>
</plugin>
```

---

## 7) WHAT IS QUALITY GATE

---

Quality Gate is a set of rules that determines whether the code is good enough to deploy.

### Example Quality Gate

#### Condition

Coverage < 80% → FAIL

Bugs > 0 → FAIL

Vulnerabilities > 0 → FAIL

Duplications > 5% → FAIL

If conditions fail

Build fails.

---

## 8) SONARQUBE IN CI/CD PIPELINE

---

### Typical CI/CD pipeline

Developer Push Code



Git Repository



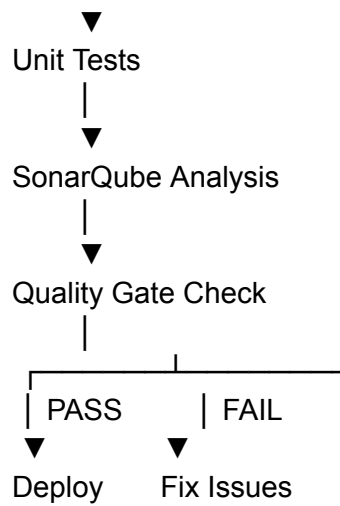
CI Tool (Jenkins / GitLab / GitHub Actions)



Build Stage

mvn clean install





---

## 9) IN WHICH PHASE SONARQUBE RUNS

---

Typically executed in

Build / Code Quality phase

Example Jenkins pipeline

Stage 1

Compile

Stage 2

Unit Tests

Stage 3

SonarQube Analysis

Stage 4

Quality Gate

Stage 5

Deploy

Example command

```
mvn clean verify sonar:sonar
```

---

## 10) COMMON ISSUES DETECTED IN REAL PROJECTS

---

### 1) Hardcoded password

```
String password = "admin123";
```

Security vulnerability

---

### 2) System.out.println

```
System.out.println("User created");
```

Bad practice

Fix

Use Logger

---

### 3) Empty catch block

```
try {
 service.call();
}
catch(Exception e) {
}
```

Issue

Exception ignored

-----

4) Duplicated code

Same method written multiple times.

-----

5) Unused variables

```
int count = 0;
```

Never used.

-----

11) SONARQUBE RULES

-----

Rules define how code is analyzed.

Example rule categories

Reliability  
Security  
Maintainability



Example Java rule

S106

Avoid using System.out

---

## 12) SONARQUBE QUALITY PROFILES

---

Quality Profile = Set of rules applied to project.

Example

Java Quality Profile

contains 1000+ rules.

Companies often create custom profiles.

---

## 13) TECHNICAL DEBT

---

Technical debt = Time required to fix issues.

Example

Issue

Large method

Estimated fix time

30 minutes

Total project technical debt

2 days

---

#### 14) SONARQUBE PULL REQUEST ANALYSIS

---

When developer creates PR

SonarQube analyzes only changed code.

Example

GitHub PR

Shows

New bugs

New vulnerabilities

Coverage drop

Helps prevent bad code merging.

---

#### 15) BEST PRACTICES IN REAL PROJECTS

---

1) Fix issues before merge

2) Maintain coverage > 80%

3) Avoid code duplication

4) Follow coding standards

## 5) Enforce Quality Gates

---

## 16) INTERVIEW ONE-LINE ANSWER

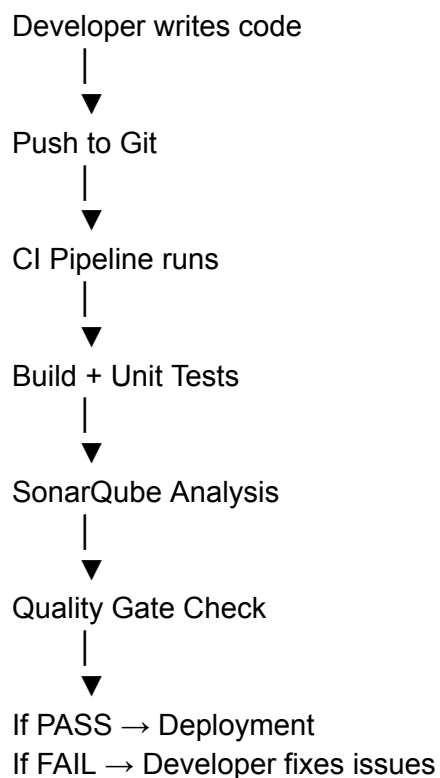
---

SonarQube is a static code analysis platform that analyzes source code during the build process to detect bugs, security vulnerabilities, code smells, duplication, and enforce quality gates to maintain high code quality.

---

## 17) REAL PROJECT FLOW SUMMARY

---



# unit and Integration testing



Scenario based

## 1 ) Self Invocation Problem in Spring @Transactional

### Problem

-----

When a method annotated with @Transactional is called from another method within the same class, the transaction may not be applied.

### Example

-----

@Service

```
public class PaymentService {
```

```
 @Transactional
```

```
 public void methodA() {
```

```
 // DB operation
```

```
 }
```

```
 public void methodB() {
```

```
 methodA(); // internal call
```

```
 }
```

```
}
```

If methodB() is called, the transaction on methodA() will NOT start.

### Reason

-----

Spring implements @Transactional using proxy-based AOP.

When a method is called from outside the bean:

Client → Spring Proxy → Target Method

The proxy intercepts the call and starts the transaction.

But when a method inside the same class calls another method:

methodB() → this.methodA()

The call bypasses the proxy and directly invokes the method.

Since the proxy is skipped, Spring cannot apply transactional logic.

### Solution

-----

1. Move the transactional method to another service class.

@Service

```
class TransactionService {
```

```
@Transactional
public void methodA() {}
}
```

2. Call the method through another bean (proxy).

```
@Autowired
TransactionService txService;

txService.methodA();
```

Key Point

-----

Spring creates a proxy for the bean (class).

Transactional behavior is applied only when the method call goes through the proxy.

Internal method calls bypass the proxy, so `@Transactional` is not applied.

## 2) Default Method Conflict in Multiple Interfaces (Diamond Problem)

Issue

When a class implements two interfaces that both contain the same default method with the same signature, Java cannot decide which implementation should be used.

Reason

Both interfaces provide their own default implementation of the same method, which creates ambiguity for the implementing class.

Example

```
interface A {
 default void m1() {
 System.out.println("A m1()");
 }
}
```

```
interface B {
 default void m1() {
 System.out.println("B m1()");
 }
}
```

```
class Test implements A, B {}
```



### Compilation Error

class Test inherits unrelated defaults for m1() from types A and B

### Solution

The implementing class must override the method and explicitly choose which interface method to use.

class Test implements A, B {

```
 @Override
 public void m1() {
 A.super.m1(); // or B.super.m1()
 }
}
```

### Key Point

If multiple interfaces provide the same default method, the implementing class must override it to resolve the conflict.

## 3) LazyInitializationException (Hibernate / JPA)

### Problem

-----

LazyInitializationException occurs when Hibernate tries to load a lazily fetched entity after the Hibernate session has already been closed.

### Error Example

-----

org.hibernate.LazyInitializationException:  
could not initialize proxy - no session

### Reason

-----

In JPA, relationships are often configured with LAZY loading.

### Example:

```
@Entity
class Order {

 @Id
 Long id;

 @ManyToOne(fetch = FetchType.LAZY)
 Customer customer;
}
```

When Order is fetched, Hibernate loads only the Order data:

```
SELECT * FROM orders WHERE id = ?
```

The 'customer' field is not loaded immediately. Instead Hibernate creates a proxy object for Customer.

Later when the code accesses:

```
order.getCustomer().getName()
```

Hibernate tries to run another query:

```
SELECT * FROM customer WHERE id = ?
```

If the Hibernate session is already closed at this time, Hibernate cannot execute the query and throws `LazyInitializationException`.

Solutions

-----

#### 1. JOIN FETCH

Load related entities in the same query.

```
@Query("SELECT o FROM Order o JOIN FETCH o.customer WHERE o.id = :id")
```

#### 2. DTO Projection

Fetch only required fields and return a DTO instead of entity.

#### 3. @Transactional

Keep the Hibernate session open so lazy properties can be loaded within the transaction.

Best Practice

-----

In production systems, DTO projection or JOIN FETCH is preferred to avoid lazy loading issues and unnecessary database queries.

## 4) Memory Leak in Spring Boot

Scenario

-----

Application memory usage keeps increasing over time and does not decrease even after requests are completed. Eventually the application may crash with `OutOfMemoryError`.

Example Error

-----  
java.lang.OutOfMemoryError: Java heap space

A memory leak happens when objects are no longer needed but are still being referenced, so the JVM Garbage Collector cannot free them.

-----

## 1. Caching Without Eviction

### Problem

-----

If caching is used but old entries are never removed, the cache keeps growing and consumes more memory.

Example:

```
@Cacheable("products")
public Product getProduct(Long id) {
 return productRepository.findById(id).get();
}
```

If thousands of products are requested, the cache keeps storing new objects without removing old ones.

### Result

-----

Memory usage keeps increasing.

### Solution

-----

Use cache eviction or TTL (time to live).

Example:

```
@CacheEvict(value = "products", key = "#id")
public void updateProduct(Product product) {
 productRepository.save(product);
}
```

Or configure cache expiration in Redis/EhCache.

-----

## 2. Static Collections

### Problem

-----

Static variables live for the entire lifecycle of the application.

Example:

```
public class UserStore {

 static List<User> users = new ArrayList<>();

}
```

If the application keeps adding users:

```
users.add(newUser);
```

The list keeps growing and objects are never removed.

Result

-----

Garbage Collector cannot remove these objects because static references still exist.

Solution

-----

Avoid static collections for storing runtime data.  
Use proper cache frameworks with eviction policies.

-----

### 3. Thread Pools Not Closed

Problem

-----

If thread pools are created but not shut down, threads remain active and hold memory references.

Example:

```
ExecutorService executor = Executors.newFixedThreadPool(10);

executor.submit(() -> processTask());
```

If the executor is never closed:

```
executor.shutdown();
```

Threads remain alive and memory usage increases.

Solution

-----

Always properly shut down thread pools.

Example:

```
executor.shutdown();
```

Or use Spring-managed thread pools such as:

```
@Async
TaskExecutor
ThreadPoolTaskExecutor
```

-----

#### 4. Large Objects Stored in Session

Problem

-----

Storing large objects in HTTP session keeps them in memory for a long time.

Example:

```
session.setAttribute("reportData", hugeList);
```

If many users store large data, memory usage grows.

Solution

-----

Store minimal data in session or move data to database/cache.

-----

#### 5. Unclosed Resources

Problem

-----

Not closing resources like streams, files, or DB connections can also increase memory usage.

Example:

```
InputStream input = new FileInputStream(file);
```

If not closed, resources remain allocated.

Solution

-----

Use try-with-resources.

Example:

```
try(InputStream input = new FileInputStream(file)) {
 // process
}
```

---

## Interview Summary

---

Memory leak in Spring Boot occurs when objects remain referenced and cannot be garbage collected. Common causes include unbounded caching, static collections, unclosed thread pools, large session objects, and unclosed resources. Proper resource management and cache eviction strategies help prevent it.

## 5) N+1 Query Problem (Hibernate / JPA)

### Definition

---

The N+1 query problem occurs when an application executes 1 query to fetch parent records and then executes additional queries (one per record) to fetch related entities.

This results in excessive database calls and poor performance.

---

### Example Scenario

Entities:

```
@Entity
class Customer {

 @Id
 Long id;

 @OneToMany(mappedBy = "customer", fetch = FetchType.LAZY)
 List<Order> orders;
}
```

```
@Entity
class Order {
```

```
@Id
Long id;

@ManyToOne
Customer customer;
}
```

---

## Code Example

Repository:

```
List<Customer> customers = customerRepository.findAll();
```

Then accessing orders:

```
for(Customer c : customers){
 System.out.println(c.getOrders().size()); //when we did this automatically extra query generated to
 Get related lazy data
}
```

---

## What Happens Internally

Step 1: Hibernate loads all customers

SQL Query 1

```
SELECT * FROM customer;
```

Suppose there are 5 customers.

Step 2: When orders are accessed for each customer

Hibernate executes additional queries:

```
SELECT * FROM orders WHERE customer_id = 1
SELECT * FROM orders WHERE customer_id = 2
SELECT * FROM orders WHERE customer_id = 3
SELECT * FROM orders WHERE customer_id = 4
SELECT * FROM orders WHERE customer_id = 5
```

Total queries executed:

1 + 5 = 6 queries

This is called the N+1 query problem.

---

### Why It Happens

Because of lazy loading. Hibernate loads related entities only when they are accessed, so each access triggers a new database query.

---

### Performance Impact

If there are 1000 customers:

1 query for customers  
1000 queries for orders

Total queries = 1001

This causes:

- high database load
  - slow API responses
  - poor scalability
- 

### Solution 1: JOIN FETCH

Load parent and child entities in a single query. And call this method from service

```
@Query("SELECT c FROM Customer c JOIN FETCH c.orders")
List<Customer> findCustomersWithOrders();
```

SQL generated:

```
SELECT c.*, o.*
FROM customer c
JOIN orders o ON c.id = o.customer_id
```

---

### Solution 2: DTO Projection

Fetch only required fields.

```
@Query("""
```



```
SELECT new com.example.CustomerDTO(c.id, c.name, o.id)
FROM Customer c
JOIN c.orders o
""")
```

This avoids entity loading and improves performance.

-----

Solution 3: EntityGraph

```
@EntityGraph(attributePaths = {"orders"})
List<Customer> findAll();
```

This tells Hibernate to fetch the orders along with customers.

-----

Interview Summary

-----

N+1 query problem occurs when one query fetches parent records and additional queries are executed for each related entity due to lazy loading.

Solutions include JOIN FETCH, DTO projections, and EntityGraph to load related data efficiently in fewer queries.

## 6) Using Retry and Circuit Breaker Together in Microservices

Scenario

-----

In a microservice architecture, Service A may call another service (Service B). If Service B is temporarily unavailable or slow, the request may fail.

To make the system more resilient, we can apply both Retry and Circuit Breaker from Resilience4j on the same method.

Retry

-----

Retry attempts the request again if the first attempt fails. This helps handle temporary failures such as network issues or short service downtime.

Circuit Breaker

-----

Circuit breaker monitors failures. If failures exceed a threshold, the circuit opens and stops further calls to the failing service. Instead, a fallback response is returned.

Using both together ensures:

- Retry handles temporary failures.
- Circuit breaker protects the system from repeated failures.

#### Execution Flow

-----

Client



Service A



Retry attempts (multiple calls if failure)



Circuit breaker monitors failure rate



If failure threshold exceeded → circuit opens



Fallback method executed

#### Example Code

-----

@Service

```
public class PaymentService {
```

```
 @Retry(name = "paymentService")
```

```
 @CircuitBreaker(name = "paymentService", fallbackMethod = "fallbackPayment")
```

```
 public Payment getPayment(Long id) {
```

```
 return paymentClient.getPayment(id);
```

```
 }
```

```
 public Payment fallbackPayment(Long id, Exception ex) {
```

```
 return new Payment("Payment service temporarily unavailable");
```

```
 }
```

```
}
```

In this example:

- Retry will attempt the request multiple times.
- If failures continue, the circuit breaker may open.
- If the circuit is open, the fallback method will execute.

#### Configuration (application.yml)

-----

```
resilience4j:
```

```
 retry:
```

```
 instances:
```

```
paymentService:
 maxAttempts: 3
 waitDuration: 1s
```

```
circuitbreaker:
 instances:
 paymentService:
 failureRateThreshold: 50
 slidingWindowSize: 10
 waitDurationInOpenState: 10s
```

## Configuration Meaning

-----

### Retry Configuration

- maxAttempts: Maximum number of retry attempts.
- waitDuration: Time delay between retry attempts.

### Circuit Breaker Configuration

- failureRateThreshold: Percentage of failures that will open the circuit.
- slidingWindowSize: Number of calls used to calculate failure rate.
- waitDurationInOpenState: Time the circuit remains open before trying again.

### Benefits

-----

- Improves fault tolerance in microservices.
- Prevents cascading failures.
- Provides fallback responses when services are unavailable.

### Best Practice

-----

In production microservices systems, resilience patterns often combine:

Timeout + Retry + Circuit Breaker + Fallback

to ensure stable and reliable service communication.

## 7) REST API Design Issue

### Problem:

API returns HTTP 200 OK even when an error occurs. This is incorrect REST design because HTTP status codes should represent the result of the request.

### Reason:

Spring Boot returns 200 by default if the controller method completes successfully and no exception is thrown. Some developers also manually return `ResponseEntity.ok()` even in error cases.

Solution:

Use proper HTTP status codes.

400 Bad Request → Invalid client input

404 Not Found → Resource not found

500 Internal Server Error → Server failure

Use `@ControllerAdvice` to implement global exception handling and return consistent error responses across the application.

## 8) Production Debugging Scenario – Random 500 Errors

Problem:

API randomly returns HTTP 500 Internal Server Error in production.

Possible causes:

Database issues, external service failures, infrastructure problems, or code exceptions.

Debugging steps:

1. Check application logs to identify stack trace and error.
2. Trace request using correlation ID across microservices.
3. Monitor database queries and connection pool usage.
4. Verify external services and APIs.
5. Check system metrics like CPU, memory, and container health.
6. If issue started after deployment, rollback to previous stable release.

=====  
SPRING BOOT INTERVIEW PREPARATION NOTES  
(3–5 Years Java / Spring Boot Developer)  
=====

=====  
SECTION 1 — 40 SPRING BOOT QUESTIONS (PRODUCT COMPANIES)  
=====

### CORE SPRING BOOT

1. What happens internally when a Spring Boot application starts?

Flow:

```
SpringApplication.run()
|
Create Application Context
|
Component Scan
|
Bean Creation
|
AutoConfiguration
|
Embedded Server Start (Tomcat)
```

## 2. Difference: Spring vs Spring Boot

Spring

- Requires manual configuration
- XML or Java config
- External server setup

Spring Boot

- Auto configuration
- Embedded server
- Starter dependencies

## 3. What is AutoConfiguration?

Spring Boot automatically configures beans based on:

- classpath dependencies
- properties configuration

Example:

If spring-web dependency exists

→ DispatcherServlet auto configured.

## 4. @SpringBootApplication internally

@SpringBootApplication =

@Configuration

@EnableAutoConfiguration

@ComponentScan

## 5. Difference between stereotypes

@Component  
Generic bean

@Service  
Business layer bean

@Repository  
DAO layer bean + exception translation

@Controller  
MVC controller

## 6. What is Spring Boot Starter?

Predefined dependency bundle.

Example:

spring-boot-starter-web

Includes  
- spring-web  
- jackson  
- tomcat

## 7. application.properties vs application.yml

properties

```
server.port=8080
spring.datasource.url=jdbc:mysql://
```

yaml

```
server:
 port: 8080
spring:
 datasource:
 url: jdbc:mysql://
```

## 8. Component Scanning

Spring scans packages from the main class package.

@SpringBootApplication  
|  
scan base package  
|  
create beans for annotated classes

---

## BEAN LIFECYCLE

### 9. Spring Bean Lifecycle

Instantiate bean  
|  
Populate dependencies  
|  
@PostConstruct  
|  
Bean ready  
|  
@PreDestroy

### 10. Singleton vs Prototype

Singleton  
- one instance per container

Prototype  
- new instance each request

### 11. @PostConstruct

Runs after bean initialization.

```
@PostConstruct
void init() {
 // initialization logic
}
```

### 12. Dependency Injection Types

Constructor Injection  
Setter Injection

Field Injection

Recommended:  
Constructor injection

---

REST API

13. @RestController vs @Controller

@RestController  
returns JSON directly

@Controller  
returns view

14. Request parameter types

@RequestParam  
query parameter

@PathVariable  
URL path parameter

@RequestBody  
JSON request body

15. Global Exception Handling

```
@ControllerAdvice
class GlobalExceptionHandler {

 @ExceptionHandler(Exception.class)
 public ResponseEntity handleException() {}

}
```

16. Validation

DTO

```
class UserDTO {
```



```
@NotNull
String name;

@email
String email;

}
```

Controller

```
public ResponseEntity create(@Valid UserDTO dto)
```

## 17. API Versioning

```
/api/v1/users
/api/v2/users
```

-----

## SPRING DATA JPA

### 18. Hibernate

ORM framework  
Maps Java objects → database tables.

### 19. Repository Types

CrudRepository  
Basic CRUD

JpaRepository  
CRUD + pagination

PagingAndSortingRepository  
pagination + sorting

### 20. Lazy vs Eager

Lazy  
data loaded when accessed

Eager  
data loaded immediately

## 21. N+1 Query Problem

1 query loads parent  
N queries load children

## 22. Dirty Checking

Hibernate automatically detects changes in entities  
and updates DB.

## 23. EntityManager

Core JPA interface to manage persistence.

-----

## TRANSACTIONS

### 24. @Transactional

Manages database transactions.

@Transactional  
public void transfer() {}

### 25. Transaction Propagation

REQUIRED  
REQUIRES\_NEW  
SUPPORTS

### 26. Isolation Levels

READ\_COMMITTED  
REPEATABLE\_READ  
SERIALIZABLE

---

## MICROSERVICES

### 27. REST Client Types

RestTemplate  
Blocking

WebClient  
Reactive

FeignClient  
Declarative REST client

### 28. API Gateway

Single entry point for microservices.

Responsibilities

- routing
- authentication
- rate limiting

### 29. Service Discovery

Services register with registry.

Example

Eureka

### 30. Circuit Breaker

Prevents cascading failures.

Example

Resilience4j

---

## PERFORMANCE

### 31. API Optimization

- DB indexing
- caching
- pagination
- async calls

### 32. Caching

Redis

```
@Cacheable("products")
public Product getProduct(Long id)
```

### 33. Large Dataset Handling

Pagination

PageRequest.of(page,size)

Streaming

Batch processing

---

## SECURITY

### 34. Spring Security Filter Chain

```
Request
|
Security Filters
|
Authentication
|
Authorization
```

### 35. JWT Authentication Flow

Client login

|

Generate JWT

|

Client sends JWT in header

|

Server validates token

### 36. Authentication vs Authorization

Authentication

who you are

Authorization

what you can access

-----

## PRODUCTION

### 37. Spring Boot Actuator

Provides endpoints

/health

/metrics

/info

### 38. Monitoring Tools

Prometheus

Grafana

Datadog

### 39. External Config

Config Server

Kubernetes ConfigMap

Secrets

## 40. Debugging Production Issues

Logs  
Metrics  
Tracing  
Rollback

## =====

## SECTION 2 — 15 SYSTEM DESIGN SCENARIOS

## =====

### 1. URL Shortener

Client  
|  
API Service  
|  
Generate hash  
|  
Store mapping  
|  
DB + Redis cache

### 2. Notification System

API  
|  
Kafka  
|  
Worker Service  
|  
Email / SMS / Push

### 3. Rate Limiter

Use the Redis counter.

Key

userId:timestamp

### 4. File Upload System

Client  
|  
Presigned URL  
|  
Upload to S3  
|  
Store metadata

## 5. Ticket Booking System

Use

Optimistic Locking  
or  
Distributed Lock

## 6. Chat Application

Client  
|  
WebSocket Server  
|  
Message Broker  
|  
Database

## 7. Logging System

Application  
|  
Logstash  
|  
Elasticsearch  
|  
Kibana

## 8. Payment System

API  
|  
Payment Service  
|  
Bank Gateway

Use

Idempotency  
Saga pattern

## 9. Audit Logging

Application

|

Kafka

|

Audit Service

|

Audit DB

## 10. Search System

Use

Elasticsearch

for full text search.

## 11. Distributed Cache

Redis Cluster

Features

TTL

Eviction

## 12. Job Scheduler

Quartz

Autosys

Cron Jobs

## 13. API Gateway Design

Gateway

|



Authentication

|

Routing

|

Rate limiting

## 14. Microservice Communication

REST

gRPC

Kafka

## 15. Multi Tenant Architecture

Shared database

Schema per tenant

Database per tenant

# =====

## SECTION 3 — REAL DEBUGGING STORIES

# =====

### Story 1 — Slow API

Situation

API response time increased to 4 seconds.

Investigation

Checked logs and DB queries.

Root Cause

Missing index.

Fix

```
CREATE INDEX idx_user_id
ON orders(user_id)
```

Result

API response reduced to 150ms.

-----

## Story 2 — Memory Leak

### Situation

Memory usage increasing continuously.

### Root Cause

Large objects cached in static map.

### Fix

Used Redis with TTL.

### Result

Memory stabilized.

---

## Story 3 — Connection Pool Exhausted

### Error

HikariPool - Connection not available

### Root Cause

Slow DB queries.

### Fix

- optimized query
- increased pool size
- added proper transaction handling

---

## Story 4 — Kafka Duplicate Processing

### Situation

Orders processed twice.

### Root Cause

Consumer retry.

### Fix

Idempotent consumer.

Store processed eventId.

---

## Story 5 — Circular Dependency

ServiceA → ServiceB

ServiceB → ServiceA

Fix

Refactored to

ServiceA

ServiceB

CommonService

---

## Story 6 — Production 500 Errors

Situation

Random failures.

Investigation

Logs

Metrics

External API latency

Fix

Added

Circuit Breaker

Retry

Timeout

---

BEST WAY TO ANSWER DEBUGGING QUESTIONS

=====

- 1 Identify the issue
- 2 Check logs and metrics
- 3 Find root cause
- 4 Fix problem
- 5 Prevent future occurrence

=====

END OF NOTES

=====

maven

Maven Command	What It Does	What Happens Internally	Output
<code>mvn clean</code>	Deletes previous build files	Removes <code>target/</code> directory	Fresh build environment
<code>mvn validate</code>	Checks project structure	Validates <code>pom.xml</code> and project configuration	Ensures project is correct
<code>mvn compile</code>	Compiles source code	Converts <code>.java</code> → <code>.class</code>	<code>target/classes</code>
<code>mvn test</code>	Runs unit tests	Executes tests using <b>JUnit / Surefire plugin</b>	Test reports
<code>mvn package</code>	Packages application	Compiles + tests + creates artifact	<code>.jar</code> / <code>.war</code> in <code>target/</code>
<code>mvn verify</code>	Runs additional checks	Integration tests, quality checks	Verified build
<code>mvn install</code>	Installs artifact locally	Copies artifact to local Maven repo	<code>~/.m2/repository</code>
<code>mvn deploy</code>	Deploys artifact to remote repo	Uploads JAR to Nexus/Artifactory	Available for other projects
<code>mvn spring-boot:run</code>	Runs Spring Boot application	Starts embedded server	Application starts
<code>mvn dependency:tree</code>	Shows dependency hierarchy	Displays all project dependencies	Dependency graph
<code>mvn dependency:analyze</code>	Finds unused dependencies	Detects redundant libraries	Optimization info
<code>mvn sonar:sonar</code>	Runs SonarQube analysis	Checks bugs, code smells, coverage	Sonar dashboard report
<code>mvn clean install</code>	Clean + build + install	Deletes old build and installs new artifact	Local repository artifact
<code>mvn clean package</code>	Clean + build jar	Rebuilds project completely	Executable jar

microservice scenario

=====

how we implement rate limiting for particular user

=====

redis rate limiter config in properties

bucket 4j

=====

Micro vs mono arch:

=====

- 1) Architecture (as per business function they divide in module in micro )
- 2) tech stack (each module can have its own diff stack)
- 3) deployment (independently deploy )
- 4) use case (smaller app use mono, large complex flexible move toward micro)
- 5) scalability (vertical in mono ,horizontal in micro)

=====

key feature of microservice

=====

- 1) independent deployment
- 2) fault isolation
- 3) scalability

=====

component of microservices

=====

core char::

- 1) service independence
- 2) domain driven design (DDD)
- 3) API communication
- 4) Technology Diversity

service registry and discovery:

- 1) Api gateway
- 2) load balancer
- 3) data management
- 4) monitoring and logging
- 5) ci/cd pipeline

=====

API gateway

=====



- 1) centralise endpoint for managing client interaction. It simplifies communication by routing.
- 2) load balancing
- 3) authentication and authorization

=====

service discovery

=====

-> enable microservices to dynamically locate and communicate with each other within a distributed system

-> connect each other services without hard coded

=====

what is container

=====

lightweight, standalone unit that bundles an app along with its dependencies, including libraries, config and runtime

=====

how do microservice communicate with each other

=====

- 1) msg queue -> aws sqs , kafka , rabbitmq
- 2) grpc
- 3) synchronise -> rest

how would you handle microservice failure

=====

-> one failure lead to cascade failure

-> circuit breaker

-> retries and fallback

-> centralize logging

how can you ensure backward compatibility

=====

- 1) api versioning
- 2) semantic versioning principles

what is OAuth and why it is used ?

=====

what is cQRS pattern in microservices ? what prob it solve?

=====

compare api gateway and load balancer in microservices?

=====

What are some common design principles?

=====

- 1) high cohesion
- 2) autonomous
- 3) business domain centric
- 4) resilience
- 5) observable
- 6) automation

how do ensure security in microservices

=====

- 1) secure the communication
- 2) authentication and authorization
- 3) service level isolation

what are the few way to achive synchronous and asynchronous communication between microservice?

=====

=====

sync -> rest api calls,grpc,graphql api

how do you implement rate limiting in microservices?

=====

- > token bucket algoritham
- > API gateway

What is an event driven arch in microservices?

=====

microservice communicate on event msgs

what is idempotency and why it is imp in microservices

=====

-> multiple request -> same type of response

what is database sharding and why used it is used in microservices?

=====

-> large database are divided into smaller more manageable pieces called "shard".

how do you handle data consistency in microservices

=====

- 1) saga pattern
- 2) event sourcing
- 3) distributed transaction

what is canary deployment in microservices?

=====

how would you monitor the microservices?

=====

- 1) distribution tracing
- 2) metrics collection (prometheus)
- 3) visualization and alerting (graphana and kibana)

what are DDD principles ?

=====

- 1) bounded context
- 2) ubiquitous languages
- 3) aggregates

how do you design for scalability in microservices?

=====

- 1) horizontal scaling
- 2) database partitioning (sharding)
- 3) caching
- 4) load balancing
- 5) asynchronous processing
- 6) service mesh

what is chaos engineering and how is it applied in microservice ?

=====

how do you perform logging in a distributed microservices system?

=====

- 1) centralized logging
- 2) structured logging
- 3) trace ids
- 4) log levels
- 5) security

what is the purpose of retry pattern in microservices

=====

what are sidecars in microservices?

=====

what are the best practices to deploy microservices in prod ?

=====

- 1) containerisation
- 2) ci/cd
- 3) deployment strategies
- 4) monitoring and observability
- 5) service discovery
- 6) security

java classic

we have two employee class which has two object both have same id and name ,now we have one set,now we want only unique employee elements to be stored in set

```
Employee e1 = new Employee(1, "Tejas");
Employee e2 = new Employee(1, "Tejas");
```

```
Set<Employee> set = new HashSet<>();
set.add(e1);
set.add(e2);
```

```
System.out.println(set.size());
```

```
class Employee {
 private int id;
 private String name;

 public Employee(int id, String name) {
 this.id = id;
 this.name = name;
 }

 // 1. Override equals
 @Override
 public boolean equals(Object o) {
 if (this == o) return true;
 if (!(o instanceof Employee)) return false;

 Employee e = (Employee) o;
 return id == e.id &&
 Objects.equals(name, e.name);
 }

 // 2. Override hashCode
 @Override
 public int hashCode() {
 return Objects.hash(id, name);
 }
}
```

Used\_in\_project

- 1) Component used in project
- 2) Config classes
- 3) Service
- 4) Repo
- 5)



Prod Defect

In our application, we use a **pipeline-based design pattern** where business logic is executed through a sequence of processing steps. A key part of this pipeline is a **mapper layer**, where we maintain precomputed logic for calculating risk scores. These risk calculations can be triggered from multiple entry points depending on the flow (e.g., assessment, self-assessment, etc.).

#### **Problem:**

Recently, a defect was introduced when an additional validation was added to ensure that **pre-calculation of risk should only be executed in an “assessment” context**.

However, this change unintentionally blocked the risk pre-calculation when the flow originated from the **self-assessment module**. As a result:

- After submitting a self-assessment, expected risk calculations were not triggered.
- This caused incorrect or missing risk scores for certain business rules.

#### **Root Cause:**

The newly added context check was too restrictive and did not account for the **self-assessment flow**, even though it also required the same pre-calculation logic.

#### **Solution:**

To fix this:

- I updated the **service layer (adapter class)** where the pipeline is orchestrated.
- Introduced conditional checks to correctly identify when recalculation should happen.
- Explicitly invoked the **mapper’s pre-calculation logic** for the self-assessment flow by passing the appropriate context.
- Ensured that the logic remains reusable and does not violate the original design constraints.

#### **Outcome:**

- Risk calculation now correctly triggers for both **assessment and self-assessment flows**.
- No regression in existing assessment behavior.
- Maintained separation of concerns by handling context-specific logic at the adapter/service layer instead of modifying core mapper logic.

#### **Key Takeaways (for interview emphasis):**

- Careful handling of **context-driven logic in pipeline systems**.
- Importance of identifying **side effects of restrictive conditions**.
- Fix applied at the right abstraction layer to avoid breaking existing design.

# Observability

# 1. kubectl — Day-to-Day Usage (EKS / Kubernetes)

## What it is

CLI to interact with Kubernetes clusters (including **Amazon EKS**).

# kubectl + Datadog — Day-to-Day Developer Guide (EKS)

```
=====
SECTION 1: kubectl (Kubernetes CLI)
=====
```

# Mental Model

Kubernetes resources you interact with:

- Pod → running container
- Deployment → manages pods (replicas, rollout)
- Service → networking
- ConfigMap → config
- Secret → sensitive config

```

1. Setup (EKS)

```

# Configure kubectl for EKS cluster

aws eks update-kubeconfig --region <region> --name <cluster-name>

# Verify connection

kubectl get nodes

```

2. Basic Commands (Most Used Daily)

```

# List pods

kubectl get pods -n <namespace>

# Wide output (IP, node, etc.)

kubectl get pods -o wide

# List deployments

kubectl get deployments

# List all resources  
kubectl get all

---

### # 3. Debugging Pods (CRITICAL)

---

# Describe pod (events + errors)  
kubectl describe pod <pod-name>

# Common things to check:  
# - CrashLoopBackOff  
# - ImagePullBackOff  
# - OOMKilled  
# - Liveness probe failure

---

### # 4. Logs (VERY FREQUENT)

---

# View logs  
kubectl logs <pod-name>

# Follow logs (like tail -f)  
kubectl logs -f <pod-name>

# Multi-container pod  
kubectl logs <pod-name> -c <container-name>

# Previous crash logs  
kubectl logs <pod-name> --previous

---

### # 5. Exec into Pod (DEEP DEBUGGING)

---

kubectl exec -it <pod-name> -- /bin/sh

# Inside container:  
# - check env vars  
# - test internal APIs  
# - inspect files

---

### # 6. Restart / Fix Deployment

---

# Restart deployment  
kubectl rollout restart deployment <deployment-name>

# Scale deployment  
kubectl scale deployment <deployment-name> --replicas=3

# Delete pod (auto recreated)  
kubectl delete pod <pod-name>

---

## # 7. Apply Changes

---

# Apply config  
kubectl apply -f deployment.yaml

# Delete resource  
kubectl delete -f deployment.yaml

---

## # 8. Port Forwarding (Local Debugging)

---

kubectl port-forward pod/<pod-name> 8080:8080

# Access locally:  
# http://localhost:8080

---

## # 9. Common Failure Cases (EDGE CASES)

---

# Pod stuck in Pending  
→ Node capacity issue / scheduling issue

# CrashLoopBackOff  
→ App crashing repeatedly

# OOMKilled  
→ Memory limit exceeded

# ImagePullBackOff  
→ Wrong image / auth issue

# No logs  
→ Container not starting

---

## # 10. Typical Debug Flow (kubectl)

---

```
kubectl get pods
kubectl describe pod <pod>
kubectl logs <pod>
kubectl exec -it <pod> -- /bin/sh
```

---

## SECTION 2: Datadog (Observability)

---

### # Signals in Datadog

- Logs → What happened
- Metrics → System health
- Traces → Request flow

---

## # 1. Logs (MOST USED)

---

### # Search queries:

```
service:payment status:error
service:user-api @http.status_code:500
trace_id:abc123
```

### # Use cases:

- Debug exceptions
- Find request failures
- Investigate production bugs

---

## # 2. Metrics (SYSTEM HEALTH)

---

### # Common metrics:

- CPU usage
- Memory usage
- Request latency (p95, p99)
- Error rate

### # Example:

```
"Is latency increasing?"
"Is CPU spiking?"
```

---

## # 3. APM (Distributed Tracing)

---

# Request flow:

API → service A → service B → DB

# Identify:

- Slow service
- Bottleneck
- External dependency latency

---

#### # 4. Alerts

---

# Examples:

- error\_rate > 5%
- latency > threshold
- pod restarts

# Notification:

- Slack
- Email

---

#### # 5. Service Map

---

# Shows:

- All services
- Dependencies
- Traffic flow

---

#### # 6. Typical Debug Flow (Datadog)

---

1. Alert triggered
2. Open logs → find error
3. Extract trace\_id
4. Open trace → find slow service
5. Check metrics → CPU/memory

---

### SECTION 3: Combined Workflow (REAL WORLD)

---

# Scenario: Production Issue

Step 1:



## Datadog Alert Triggered

Step 2:

Check logs in Datadog

→ Identify error

Step 3:

Check trace

→ Identify failing service

Step 4:

Check metrics

→ CPU spike? latency?

Step 5:

If deeper issue:

kubectl get pods

kubectl describe pod

kubectl logs

kubectl exec

Step 6:

Fix issue

→ restart / deploy

## =====

## SECTION 4: Responsibility Split

## =====

# Datadog

- Observability
- Logs
- Metrics
- Traces
- Alerts

# kubectl

- Control
- Debug infra
- Restart pods
- Deploy changes

## =====

## SECTION 5: What You Use Daily

## =====

# Very Frequent

- Datadog logs

- Datadog dashboards
- kubectl logs
- kubectl describe

#### # Moderate

- kubectl exec
- Datadog tracing

#### # Less Frequent

- kubectl apply
- scaling deployments

## =====

## FINAL SUMMARY

## =====

Datadog → Detect + Analyze issues

kubectl → Fix + Control system

Both are required for real-world Kubernetes workflows.

#### # kubectl + Datadog — Common Interview Q&A (Developer Focus)

## =====

## SECTION 1: FUNDAMENTALS

## =====

Q1. What is kubectl?

A:

kubectl is the CLI used to interact with Kubernetes clusters.

It allows you to deploy, inspect, debug, and manage resources like pods, deployments, and services.

-----

Q2. What is Datadog?

A:

Datadog is an observability platform that provides:

- Logs
- Metrics
- Distributed tracing (APM)
- Alerts

It helps monitor and debug applications in production.

-----

Q3. Difference between kubectl and Datadog?

A:

kubectl → Control plane (operate cluster)

Datadog → Observability (monitor/debug system)

kubectl = "fix"

Datadog = "detect + analyze"

-----

Q4. Can Datadog replace kubectl?

A:

No.

Datadog cannot:

- Restart pods
- Exec into containers
- Apply deployments

kubectl is mandatory for cluster operations.

=====

## SECTION 2: REAL-WORLD DEBUGGING

=====

Q5. How do you debug a production issue?

A:

1. Datadog alert triggers
2. Check logs in Datadog
3. Identify error / trace\_id
4. Use APM to trace request flow
5. Check metrics (CPU, memory, latency)
6. If needed:
  - kubectl get pods
  - kubectl describe pod
  - kubectl logs
  - kubectl exec
7. Fix and redeploy

-----

Q6. Pod is in CrashLoopBackOff. What do you do?

A:

kubectl describe pod <pod>

kubectl logs <pod> --previous

Check:

- Application crash

- Wrong config
  - Missing env variables
- 

Q7. Logs not visible in Datadog. What next?

A:

Use `kubectl logs <pod>`

Possible issues:

- Datadog agent not running
  - Log pipeline misconfigured
  - Delay in ingestion
- 

Q8. How do you check if service is slow?

A:

Use Datadog APM:

- Trace request
- Identify slow service
- Check latency breakdown

Then correlate with:

- CPU/memory metrics
- 

Q9. How do you debug high latency?

A:

1. Datadog APM → find slow span
2. Check downstream dependencies
3. Verify DB/external calls
4. If infra issue:
  - `kubectl top pods`
  - `kubectl describe pod`

---

### SECTION 3: kubectl DEEP QUESTIONS

---

Q10. Difference between get, describe, logs?

A:

`kubectl get` → list resources (high-level)

kubectl describe → detailed info + events  
kubectl logs → application logs

---

Q11. How do you exec into a pod?

A:  
kubectl exec -it <pod> -- /bin/sh

Used for:  
- Debugging runtime issues  
- Checking files/config

---

Q12. How do you restart a pod?

A:  
kubectl delete pod <pod>

OR

kubectl rollout restart deployment <deployment>

---

Q13. What happens when you delete a pod?

A:  
If managed by deployment:  
→ Kubernetes recreates it automatically

---

Q14. How do you scale an application?

A:  
kubectl scale deployment <name> --replicas=5

---

Q15. How do you apply changes?

A:  
kubectl apply -f deployment.yaml

---

## SECTION 4: DATADOG DEEP QUESTIONS

Q16. What are logs, metrics, and traces?

A:

Logs → discrete events (errors, messages)

Metrics → numeric time-series (CPU, latency)

Traces → request journey across services

Q17. What is APM?

A:

Application Performance Monitoring:

Tracks requests across microservices and shows latency breakdown.

Q18. How does Datadog collect data?

A:

- Agent installed on host/container
- APM libraries in application
- Integrations (Kubernetes, AWS)

Q19. What is a trace\_id?

A:

Unique ID for a request across services.

Used to correlate logs, metrics, and traces.

Q20. How do you create alerts?

A:

Based on:

- Metrics (CPU, latency)
- Logs (error count)
- APM (error rate)

## SECTION 5: SCENARIO-BASED QUESTIONS

Q21. Pod is running but API is failing. Why?

A:

Possible reasons:

- Application bug
- Wrong config
- Downstream service failure

Check:

Datadog logs → trace → kubectl logs

---

Q22. High CPU usage in pods. What do you do?

A:

1. Datadog metrics → confirm spike
  2. kubectl top pods
  3. Check logs
  4. Scale or optimize
- 

Q23. How do you check environment variables in a pod?

A:

kubectl exec -it <pod> -- printenv

---

Q24. How do you debug networking issues?

A:

kubectl exec into pod  
curl internal service

Also check:

- service config
  - DNS
- 

Q25. When do you use kubectl over Datadog?

A:

- Need to modify cluster
- Debug container internals

- Restart or deploy services

=====

## SECTION 6: TRICK / EDGE CASE QUESTIONS

=====

Q26. Why logs differ between Datadog and kubectl?

A:

- Datadog ingestion delay
  - Filtering rules
  - Misconfigured agent
- 

Q27. Why pod has no logs?

A:

- Container crashes immediately
  - Logging not configured
  - Wrong container selected
- 

Q28. Multiple containers in pod—how to get logs?

A:

kubectl logs <pod> -c <container>

-----

Q29. Pod stuck in Pending?

A:

- No node capacity
  - Resource constraints
  - Taints/tolerations issue
- 

Q30. OOMKilled means?

A:

Container exceeded memory limit and was killed.

=====

## FINAL SUMMARY

=====



kubectl → execution + control  
Datadog → visibility + analysis

Interview key:

Always explain flow:

"Detect in Datadog → Debug deeper with kubectl → Fix"

database scenario github



# 12 Factor of Microservice

## 1. Domain Decomposition (DDD)

- Use Domain-Driven Design
  - Identify **bounded contexts** and split services around business capabilities
  - Avoid technical-layer-based splitting (e.g., service per DAO)
- 

## 2. Single Responsibility per Service

- Each service should own **one business capability**
  - Reduces coupling and simplifies scaling + maintenance
- 

## 3. Loose Coupling & High Cohesion

- Services must be independent
  - Avoid shared databases
  - Communicate via APIs/events only
- 

## 4. Database per Service

- Each microservice owns its data
  - No direct cross-service DB access
  - Ensures autonomy and independent deployments
- 

## 5. Inter-Service Communication

- Sync: REST / gRPC
  - Async: messaging (Kafka, RabbitMQ)
  - Prefer async for scalability and fault tolerance
- 

## 6. API Gateway

- Use a centralized entry point like Spring Cloud Gateway or NGINX
  - Handles routing, authentication, rate limiting
-

## 7. Service Discovery

- Dynamic service registration and lookup
  - Tools: Eureka, Consul
  - Required for scaling and dynamic environments
- 

## 8. Fault Tolerance & Resilience

- Circuit breakers, retries, timeouts
  - Tools: Resilience4j
  - Prevent cascading failures
- 

## 9. Observability (Monitoring + Logging + Tracing)

- Metrics: Prometheus
  - Visualization: Grafana
  - Tracing: Zipkin / Datadog
- 

## 10. CI/CD Pipeline

- Automate build, test, deploy
  - Tools: Jenkins, GitLab
  - Enables frequent and safe deployments
- 

## 11. Containerization & Orchestration

- Package services using Docker
  - Orchestrate with Kubernetes
  - Enables scaling, self-healing, portability
- 

## 12. Security

- Authentication & authorization (OAuth2, JWT)
- Secure communication (HTTPS, mTLS)
- API Gateway often enforces security policies

AWS

#### Amazon EC2

- Hosts backend services, Kafka, Redis (if self-managed)

#### Amazon EKS (Kubernetes)

- Runs microservices as containers (pods)
- Handles scaling, deployment, orchestration

#### Amazon RDS (MySQL)

- Stores audit data, users, roles, findings

#### Amazon S3

- Stores files (audit evidence, reports, attachments)

#### Amazon ElastiCache (Redis)

- Caches roles, permissions, sessions for performance

#### Amazon MSK (Kafka)

- Handles async communication between services (event-driven)

#### AWS Lambda

- Runs background jobs, scheduled tasks, lightweight processing

#### Amazon API Gateway

- Entry point for frontend → backend APIs
- Handles routing, security, throttling

#### Amazon CloudWatch

- Logs, monitoring, alerts for services



Config

## ◆ Configuration & Secret Management in Our Application

- **Kubernetes ConfigMap**

- Used for **non-sensitive configuration**
  - Examples:
    - Service URLs (other microservices endpoints)
    - Kafka topics
    - Feature flags
    - Timeout/retry configs
  - Reason: Easy to update without code change, safe to expose
- 

- **Kubernetes Secret**

- Used for **basic sensitive data inside cluster**
  - Examples:
    - Database username/password
    - JWT secret
    - Internal API keys
  - Reason: Native Kubernetes support, easy injection into pods
- 

- **AWS Secrets Manager**

- Used for **highly sensitive and critical secrets (production)**
  - Examples:
    - Production DB credentials
    - Third-party API credentials
    - Encryption keys
  - Reason:
    - Strong security (KMS encryption)
    - Secret rotation
    - IAM-based access control
- 

## ◆ Overall Approach

- ConfigMap → **non-sensitive configs**
- Kubernetes Secret → **basic secrets**
- AWS Secrets Manager → **secure, production-grade secrets**

Retry

```
<dependency>
 <groupId>io.github.resilience4j</groupId>
 <artifactId>resilience4j-spring-boot2</artifactId>
</dependency>
```

```
<!-- Optional but recommended (for monitoring) -->
```

```
<dependency>
 <groupId>io.github.resilience4j</groupId>
 <artifactId>resilience4j-micrometer</artifactId>
</dependency>
```

```
<dependency>
 <groupId>io.github.resilience4j</groupId>
 <artifactId>resilience4j-spring-boot2</artifactId>
</dependency>
```

```
<!-- Optional but recommended (for monitoring) -->
```

```
<dependency>
 <groupId>io.github.resilience4j</groupId>
 <artifactId>resilience4j-micrometer</artifactId>
</dependency>
```

Yml

resilience4j:

retry:

instances:

notificationService: # Logical name (same as @Retry(name="..."))

max-attempts: 3

# Total attempts INCLUDING first call

# Example: 3 = 1 original + 2 retries

wait-duration: 2s

# Delay between retries

# Avoid 0 → can overload system

enable-exponential-backoff: true

# If true → delay increases each retry

exponential-backoff-multiplier: 2

# Delay pattern:

# 1st retry → 2s

# 2nd retry → 4s

# 3rd retry → 8s

```
retry-exceptions:
 - java.io.IOException
 - java.net.SocketTimeoutException
Only retry for these exceptions

ignore-exceptions:
 - java.lang.IllegalArgumentException
NEVER retry these (business errors)

retry-exception-predicate: null
Advanced: custom logic to decide retry

fail-after-max-attempts: true
After all retries fail → throw exception
```

#### Example

=====

```
@Service
public class NotificationClient {

 private final RestTemplate restTemplate;

 public NotificationClient(RestTemplate restTemplate) {
 this.restTemplate = restTemplate;
 }

 @Retry(name = "notificationService", fallbackMethod = "sendFallback")
 public String sendNotification(String auditId) {

 System.out.println("Calling Notification Service...");

 // Simulating REST call
 String url = "http://notification-service/api/send/" + auditId;

 return restTemplate.getForObject(url, String.class);
 }

 // Fallback method (must have same params + Exception)
 public String sendFallback(String auditId, Exception ex) {
 System.out.println("Fallback triggered due to: " + ex.getMessage());
 return "Notification failed, fallback response";
 }
}
```